УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У НОВОМ САДУ

Лазар Адамовић

Реализација апликације за проширену стварност на *Android* платформи

ДИПЛОМСКИ РАД
- Основне академске студије -

Нови Сад, март 2026.

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, **РБР**:			
Идентификациони број, **ИБР**:			
Тип документације, **ТД**:		Монографска документација	
Тип записа, **ТЗ**:		Текстуални штампани материјал	
Врста рада, **ВР**:		Завршни (Bachelor) рад	
Аутор, **АУ**:		Лазар Адамовић	
Ментор, **МН**:		проф. др Иштван Пап	
Наслов рада, **НР**:		Реализација апликације за проширену стварност на Android платформи	
Језик публикације, **ЈП**:		Српски / латиница	
Језик извода, **ЈИ**:		Српски	
Земља публиковања, **ЗП**:		Република Србија	
Уже географско подручје, **УГП**:		Војводина	
Година, **ГО**:		2026	
Издавач, **ИЗ**:		Ауторски репринт	
Место и адреса, **МА**:		Нови Сад; трг Доситеја Обрадовића 6	
Физички опис рада, **ФО**: (поглавља/страна/ цитата/табела/слика/графика/прилога)		7/63/0/3/27/0/0	
Научна област, **НО**:		Електротехника и рачунарство	
Научна дисциплина, **НД**:		Рачунарска техника	
Предметна одредница/Кључне речи, **ПО**:		Проширена стварност, ARCore, Android, Sceneform, .NET, GPS	
УДК			
Чува се, **ЧУ**:		У библиотеци Факултета техничких наука, Нови Сад	
Важна напомена, **ВН**:			
Извод, **ИЗ**:		Предмет овог рада је пројектовање и реализација клијент-сервер система за приказ информација у проширеној стварности на Android платформи. Систем обухвата мобилну апликацију засновану на ARCore технологији и позадински сервис развијен у .NET окружењу за управљање подацима.	
Датум прихватања теме, **ДП**:		11.10.2025.	
Датум одбране, **ДО**:			
Чланови комисије, **КО**:	Председник:	проф. др Иван Каштелан	
	Члан:	проф. др Марија Антић	Потпис ментора
	Члан, ментор:	проф. др Иштван Пап	

UNIVERSITY OF NOVI SAD ● **FACULTY OF TECHNICAL SCIENCES**
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, **ANO**:	
Identification number, **INO**:	
Document type, **DT**:	Monographic publication
Type of record, **TR**:	Textual printed material
Contents code, **CC**:	Bachelor Thesis
Author, **AU**:	Lazar Adamović
Mentor, **MN**:	Ištvan Pap, PhD
Title, **TI**:	Implementation of an Augmented Reality Application on the Android Platform
Language of text, **LT**:	Serbian
Language of abstract, **LA**:	Serbian
Country of publication, **CP**:	Republic of Serbia
Locality of publication, **LP**:	Vojvodina
Publication year, **PY**:	2026
Publisher, **PB**:	Author's reprint
Publication place, **PP**:	Novi Sad, Dositeja Obradovica sq. 6
Physical description, **PD**: (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/63/0/3/27/0/0
Scientific field, **SF**:	Electrical Engineering
Scientific discipline, **SD**:	Computer Engineering, Engineering of Computer Based Systems
Subject/Key words, **S/KW**:	Augmented Reality, ARCore, Android, Sceneform, .NET, GPS
UC	
Holding data, **HD**:	The Library of Faculty of Technical Sciences, Novi Sad, Serbia
Note, **N**:	
Abstract, **AB**:	The subject of this paper is the design and implementation of a client-server system for displaying information in augmented reality on the Android platform. The system includes a mobile application based on ARCore technology and a backend service developed in the .NET environment for data management.
Accepted by the Scientific Board on, **ASB**:	11.10.2025.
Defended on, **DE**:	
Defended Board, **DB**: President:	Ivan Kaštelan, PhD
Member:	Marija Antić, PhD
Member, Mentor:	Ištvan Pap, PhD

Menthor's sign

	УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА** 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

Студијски програм:	Рачунарство и аутоматика		
Студент:	Лазар Адамовић	Број индекса:	RA 147/2020
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	проф. др Иштван Пап		

НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА:

- проблем – тема рада;
- начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;

НАСЛОВ ЗАВРШНОГ РАДА:

Реализација апликације за проширену стварност на Android платформи

ТЕКСТ ЗАДАТКА:

У оквиру рада потребно је пројектовати и реализовати мобилну апликацију на Android платформи која користи технологије проширене стварности (AR) за приказ виртуелних ознака у реалном приказу камере. Апликација треба да омогући корисницима оријентацију у простору помоћу визуелних смерница ка факултетима, ресторанима и другим тачкама од интереса (POI). У оквиру реализације потребно израдити систем заснован на Google ARCore платформи и Sceneform библиотеци, омогућити управљање тачкама од интереса кроз клијент-сервер архитектуру и административни панел, развити алгоритам за фузију сензора уз примену филтера за стабилизацију приказа, реализовати 2D мапу и извршити тестирање прецизности у реалном окружењу,

Руководилац студијског програма:	Ментор рада:
проф. др Иван Каштелан	проф. др Иштван Пап

Примерак за: o - Студента; o - Ментора

УНИВЕРЗИТЕТ У НОВОМ САДУ ● **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

проф. др Иштван Пап

_____

Кандидат

Лазар Адамовић

_____

Захвалност

Желео бих да изразим захвалност свима који су на било који начин допринели реализацији овог рада. Посебно бих се захвалио свом ментору на указаном поверењу, пруженој помоћи и корисним саветима током развоја овог система. Такође хвала мојој породици и пријатељима на подршци, разумевању и стрпљену које су ми пружали током свих година студирања.

Садржај

1. Увод 1
2. Теоријске основе 3
2.1 Проширена стварност (*Augmented Reality)* 3
2.1.1 Историја проширене стварности 4
2.1.2 Области примене проширене стварности 6
2.1.3 Негативни утицај и технички изазови проширене стварности 7
2.2 *Android* оперативни систем 7
2.2.1 Архитектура *Android* система 8
2.2.2 Животни циклус активности и управљање ресурсима 9
2.2.3 Сензорски систем и подршка физичке архитектуре 10
2.3 .NET и инфраструктура позадинског система 12
2.3.1 .NET 8 и чиста архитектура 12
2.3.2 REST архитектура и размена података 14
2.3.3 Складиштење података и ORM 14
2.4 *Google ARCore* платформа 15
2.4.1 Визуелно-инерцијална одометрија (VIO) 16
2.4.2 Детекција површина и референтне тачке (*Anchors*) 16
2.5 *Sceneform* библиотека 16
2.5.1 Хијерархија сцене 17
2.5.2 GLB формат 3D модела и PBR рендеровање 17
2.5.3 UV мапирање текстура 18
2.5.4 Визуелна стабилност и глаткоћа приказа 18
2.6 *Jetpack Compose* и декларативни приступ развоју интерфејса 19

2.6.1 Декларативни приступ и управљање стањима 19
2.6.2 Компатибилност са постојећим системима 20
2.7 Системи за приказ дигиталних мапа 20
2.7.1 *OpenStreetMap* и плочасти приказ 20
2.7.2 Карактеристике и функционалности библиотеке 21
3. Концепт решења....22
3.1 Логичка организација и структура података 23
3.2 Опис апликације и корисничког интерфејса 25
3.2.1 Модул проширене стварности 25
3.2.2 Модул интерактивне мапе 26
3.2.3 Административни модул 28
4. Програмско решење....30
4.1 Развојно окружење и конфигурација 30
4.1.1 Подешавање пројекта и *Gradle* зависности 30
4.1.2 Интеграција *ARCore* и *Sceneform* библиотеке 31
4.1.3 Конфигурација *Osmdroid* библиотеке за рад са мапама 32
4.2 Реализација корисничког интерфејса....33
4.2.1 Развој компоненти и вишеслојна архитектура приказа....33
4.2.2 Динамичко прилагођавање интерфејса и системска обавештења 34
4.2.3 Интерфејс навигационог слоја и интерактивне мапе....35
4.2.4 Административни интерфејс за управљање подацима....36
4.3 Реализација серверског система на .NET 8 платформи....37
4.3.1 Доменски модели и техничка прецизност 37
4.3.2 Апстракција приступа подацима и *Repository* образац....38
4.3.3 CQRS образац и DI механизам....39
4.3.4 Конфигурација перзистенције кроз *Fluent API* 40
4.3.5 Презентациони слој и преглед API путања....41
4.4 Реализација хибридног приказа 42
4.4.1 Реализација мрежне комуникације коришћењем *Retrofit* библиотеке 43
4.4.2 Функционалност проширене стварности 44
4.4.2.1 Управљање дозволама и приступ сензорима 44
4.4.2.2 Конфигурација *ARCore* сесије и подешавање сцене 44
4.4.2.3 Обрада података са сензора и софтверска калибрација азимута....45
4.4.2.4 Рендеровање 3D маркера у реалном времену 49
4.4.3 Реализација интерактивне мапе 51

4.4.3.1 Логика приказа и аутоматско центрирање локације 51
4.4.3.2 Креирање и динамичко ажурирање маркера 52
4.5 Реализација административног модула и управљања подацима.....53
4.5.1 Механизам прикривене аутентификације 53
4.5.2 Логика креирања нових тачака од интереса 53
4.5.3 Динамичко управљање стањима 53
5. Испитивање решења и резултати 55
5.1 Испитивање функционалности система 55
5.2 Анализа и корекције делимично успешних тестова 57
6. Закључак 59
7. Литература.....61

Списак слика

Слика 2.1 – *"The Sword of Damocles"* 4
Слика 2.2 – *Visual Fixtures* 4
Слика 2.3 – *Pokemon GO* 5
Слика 2.4 – Примена проширене стварности у медицини 6
Слика 2.5 – Архитектура *Android* оперативног система 8
Слика 2.6 – Животни циклус *Android* апликације 9
Слика 2.7 – Приказ GPS система 11
Слика 2.8 – 6DoF – *Six Degrees Of Freedom* 11
Слика 2.9 – Фузија сензора 12
Слика 2.10 – Шема CQRS обрасца 13
Слика 2.11 – Принцип чисте архитектуре 13
Слика 2.12 – Размена података путем REST архитектуре 14
Слика 2.13 – Архитектура хибридног система за управљање подацима кроз .NET Web API и *Entity Framework Core* 15
Слика 2.14 – Детекција површина и приказ карактеристичних тачака 16
Слика 2.15 – Хијерархијска структура *Sceneform* сцене 17
Слика 2.16 – Структура једног GLB фајла 17
Слика 2.17 – UV мапирање у алату *Blender* 18
Слика 2.18 – Механизам једносмерног протока података *Jetpack Compose* 19
Слика 2.19 – Хијерархијска структура плочица унутар *OpenStreetMap* система 20
Слика 3.1 – Дијаграм модула који чине решење 22
Слика 3.2 – Шема релационе базе података са дефинисаним POI објектима и категоријама 24

Слика 3.3 – Аутоматски креирани директоријуми за опис тачака од интереса.....25
Слика 3.4 – Кориснички интерфејс AR модула 26
Слика 3.5 – Приказ интерактивне мапе 27
Слика 3.6 – Изглед админ панела и форме за додавање нове локације.....28
Слика 3.7 – Дијалог за унос лозинке и приступ админ панелу 29
Слика 4.1 – Приказ структуре засноване на чистој архитектури 37

Списак табела

Табела 4.1 – Преглед имплементираних REST приступних тачака на серверу.....42
Табела 5.1 – Резултати тестирања апликације 57
Табела 5.2 – Анализа и корекција.....58

Скраћенице

AR - *Augmented Reality,* Проширена стварност

3D - *Three-Dimensional space*, Тродимензионални простор

GPS - *Global Positioning System,* Глобални позициони систем

POI - *Point of Interest,* Тачка интереса

REST - *Representational State Transfer*

API - *Application Programming Interface*, Програмска спрега за апликације

HMD - *Head-mounted display,* Екран монтиран на глави

2D - *Two-Dimensional space*, Дводимензионални простор

GPU - *Graphics Processing Unit,* Графичка процесорска јединица

CQRS - *Command Query Responsibility Segregation,* Раздвајање одговорности команди и упита

HTTP - *HyperText Transfer Protocol,* Протокол за пренос хипермедијалних докумената

JSON - *JavaScript Object Notation*, Формат за размену података заснован на *JavaScript* – у

ORM - *Object-Relational Mapper,* Објектно-релациони мапер

LINQ - *Language Integrated Query*

SDK - *Software Development Kit,* Алати за развој софтвера

LiDAR - *Light Detection and Ranging,* Ласерско даљинско мерење

VIO - *Visual-inertial odometry*, Визуелно-инерциона одометрија

6DoF - *Six degrees of freedom*, Шест степени слободе

OpenGL - *Open graphics library,* Отворена библиотека графике

glTF - *Graphics Library Transmission Format,* Формат за размену 3D сцена и модела

GLB - *glTF Binary,* бинарни *glTF*

HAL - *Hardware Abstraction Layer*, Слој хардверске апстракције

DEX - *Dalvik executable format*

PBR - *Physically Based Rendering*, Физички засновано рендеровање

XML - *Extensible Markup Language*

FPS - *Frame Per Second* , Број кадрова по секунди

NDK - *Native Development Kit ,* Алати за нативни развој

APK **-** *Android Package,* Формат у који се пакује *Android* апликација

ADB - *Android Debug Bridge* , Андроид алат за дебаговање

IDE - *Integrated Development Environment,* Интегрисано развојно окружење

CRUD - *Create, Read, Update, Delete,* Креирати, Читати, Ажурирати, Обрисати

DI - *Dependency Injection,* Убризгавање зависности

1. Увод

Савремени развој мобилних уређаја омогућио је излазак дигиталног садржаја из оквира класичних екрана у стварни физички простор путем проширене стварности (AR). Ова технологија мења начин на који корисници посматрају своју околину, нудећи им могућност да виде информације и објекте који су просторно фиксирани за стварне локације. У поређењу са виртуелном стварношћу, проширена стварност поставља специфичне изазове пред програмере, пре свега у домену прецизности сензора и стабилности приказа у динамичним спољним условима.

Мотивација за израду овог рада лежи у истраживању споја локацијских сервиса и AR технологије. Док стандардне навигационе апликације захтевају од корисника интерпретацију дводимензионалних мапа, AR приступ омогућава директну визуализацију циљева у 3D простору. Главни циљ пројекта је реализација функционалног система у виду мобилног водича кроз кампус Универзитета, који би корисницима омогућио интуитивно проналажење факултета, служби и осталих тачака од интереса. Систем је дизајниран тако да у реалном времену преузима податке са удаљеног сервера, обради географске координате и пројектује 3D маркере са минималним одступањем. Посебан фокус стављен је на решавање проблема “шума” и грешака GPS сензора који су карактеристични за рад на *Android* платформи.

Апликација је развијена коришћењем *Kotlin* програмског језика и *ARCore* библиотеке, док је систем за управљање подацима реализован кроз .NET архитектуру и SQL базу података преко *Entity Framework Core*. Овакав системски приступ омогућио је централизовано управљање тачкама од интереса и њихову тренутну ажурност за све кориснике апликације.

Овај рад је структуиран кроз седам поглавља:

- У другом поглављу постављен је теоријски оквир за реализацију пројекта. Овде су анализирани принципи технологије проширене стварности, са посебним фокусом на специфичности развоја за *Android* оперативни систем и могућности *ARCore* софтверског пакета. Поред мобилне платформе, поглавље обухвата и основе релационих база података и .NET архитектуре, који чине кичму позадинског дела система.

- Треће поглавље излаже целокупан концепт решења. Фокус је на архитектури система заснованој на дистрибуираној структури, где се објашњава на који начин подаци о тачкама интереса (POI) у кампусу путују од сервера до самог корисника. Такође, дефинисан је и механизам за визуелну репрезентацију локација унутар апликације.

- У четвртом поглављу фокус се помера на практичну имплементацију и програмски код. Детаљно су описани процеси изградње REST API сервиса, структура базе података, као и логика корисничке апликације развијене у језику *Kotlin*. Посебан део посвећен је математичкој логици за прорачун релативних позиција објеката као и процесу динамичког генерисања и рендеровања маркера унутар сцене.

- Пето поглавље обухвата фазу тестирања и анализу добијених резултата. Приказани су сценарији коришћења апликације у реалним условима универзитетског кампуса, при чему је посебна пажња посвећена мерењу стабилности приказа и поузданости преноса података у различитим мрежним условима.

- У шестом поглављу изведени су закључци на основу спроведеног рада. Обједињени су изазови са којима се систем суочио, процењена је употребна вредност апликације као водича и дат је предлог могућих надоградњи, попут интеграције са напреднијим сервисима за мапирање.

Рад се завршава седмим поглављем, у којем је наведен преглед литературе, техничке документације и научних референци који су послужили као основа за израду овог решења.

2. Теоријске основе

У овом поглављу биће описане теоријске основе које чине темељ реализације апликације у овом раду. Детаљно ће бити представљени кључни концепти проширене стварности, њен историјски развој, као и изазови који се јављају приликом коришћења ове технологије у мобилном окружењу. Поред опште теорије, поглавље обухвата преглед *Android* платформе и њеног специфичног животног циклуса, што је од суштинског значаја за ефикасно управљање ресурсима физичке архитектуре попут камере и GPS сензора. Посебан акценат ће бити на архитектури система, где ће бити објашњена улога .NET развојног оквира, као и примена *Entity Framework Core* алата за управљање релационим базама података. Такође, биће размотрена улога *Google ARCore* платформе као и примарног развојног оквира, уз осврт на значај локацијских сервиса који омогућавају функционалност и прецизност ове апликације.

2.1 Проширена стварност (*Augmented Reality)*

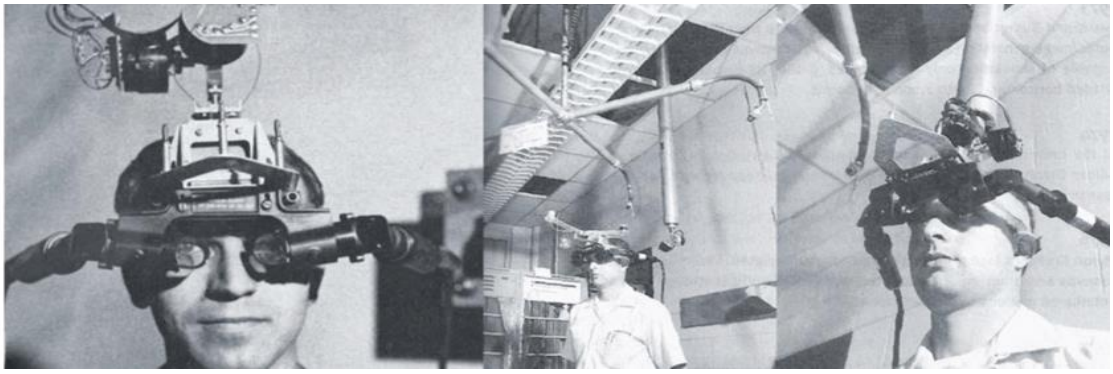
Према дефиницији "Проширена стварност представља варијацију виртуелних окружења која омогућава кориснику да види виртуелне објекте постављене преко стварног света" [1]. За разлику од виртуелне стварности, која корисника у потпуности изолује у вештачки креиран свет, проширена стварност користи постојеће физичко окружење као основу, допуњујући га дигиталним објектима у реалном времену.

Савремена проширена стварност своју најширу примену проналази путем паметних мобилних телефона, који захваљујући екранима високе резолуције омогућавају веродостојан приказ дигиталних информација убачених у видео пренос са камере [2]. Мобилни уређаји користе скуп сензора за оријентацију и позиционирање како би

израчунали релативни однос између корисника и виртуелног објекта [3]. Овај процес омогућава кориснику слободу кретања кроз физички простор док дигитални ентитети задржавају своју статичну позицију у односу на реалне координате, чиме се постиже висок степен осећаја присутности без потребе за додатном, скупом опремом [4].

2.1.1 Историја проширене стварности

Корени ове технологије сежу у 1968. годину, када је професор са Харварда, Иван Сатерланд (енгл. *Ivan Sutherland*), конструисао први систем за приказ информација на глави (HMD). Систем је назван "*The Sword of Damocles*" због своје тежине, јер је морао бити причвршћен за плафон како би пратио покрете главе корисника. Иако је могао да прикаже само једноставне линијске 3D објекте, овај уређај је поставио темеље за концепт визуелизације дигиталног садржаја преко стварног света [5].



Слика 2.1 – *"The Sword of Damocles"*

Термин проширена стварност званично су увели истраживачи компаније Боинг (енгл. *Boeing*), Том Кодел (енгл. *Tom Caudell*) и Дејвид Мајзел (енгл. *David Mizzel*), 1990. године. Они су развили систем који је радницима омогућавао да виде дигиталне шеме инсталација пројектоване директно преко стварних панела авиона, што је драстично убрзало процес производње и смањило број грешака [6]. Убрзо након тога, 1992. године, Луис Розенберг (енгл. *Louis Rosenberg*) развио је систем *Visual Fixtures* за потребе америчког ваздухопловства, што је био први потпуно функционалан систем који је кориснику омогућавао интеракцију са виртуелним алатима у физичком простору [7].

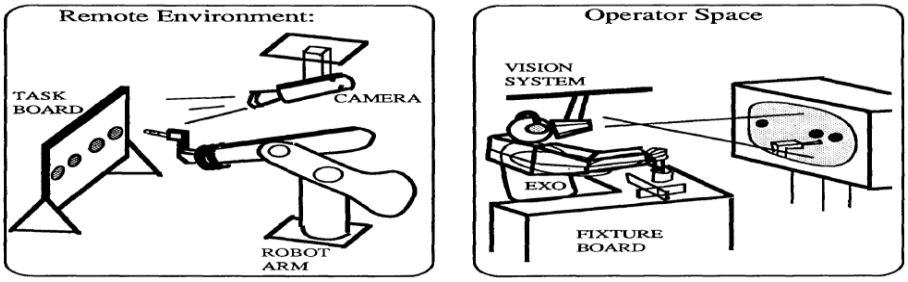


Figure 1: Experimental Setup for Telepresence Performace Assessment showing operator and workspace.

Слика 2.2 – *Visual Fixtures*

Значајан помак у доступности технологије догодио се 2000. године појавом *ARToolKit* библиотеке коју је развио Хироказу Като. Овај софтвер отвореног кода омогућио је програмерима да користе обичне веб-камере за праћење графичких маркера и постављање виртуелних објеката изнад њих. Ово је био кључни тренутак за прелазак ове технологије са скупе војне опреме на стандардне личне рачунаре [8].

Појавом паметних телефона који обједињују камере, GPS пријемнике, и напредне сензоре, AR технологија доживљава велику трансформацију. Апликација *Layar* (2009) била је једна од првих која је користила локацију корисника за приказ информација о објектима у околини, што је заправо концепт који се примењује у овом раду. Глобалну популарност донела је игра *Pokémon GO* 2016. године, која је огромном броју корисника представила концепт „лова“ виртуелних бића у стварном географском простору [9]. Захваљујући платформама као што је *Google ARCore*, у данашње време омогућено је прецизно праћење простора без потребе за маркерима, чиме је ова технологија постала саставни део навигационих и едукативних апликација [10].

Слика 2.3 – *Pokemon GO*

2.1.2 Области примене проширене стварности

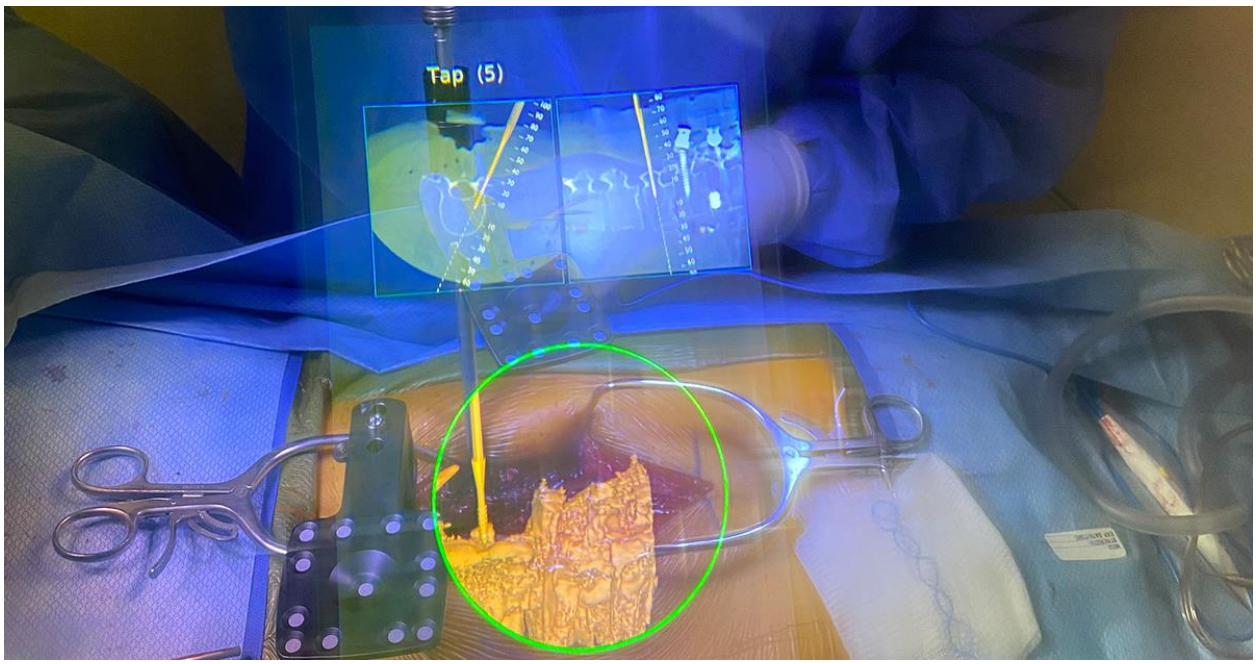
Проширена стварност проналази своју примену у готово свим сферама савременог друштва, мењајући начин на који корисници приступају информацијама и комуницирају са окружењем. Најзначајније области примене су:

Медицина – технологија омогућава хирурзима преглед унутрашњих органа или скелета пацијента директно преко његовог тела током операције користећи податке са магнетне резонанце или скенера. Ово у великој мери повећава прецизност и смањује ризик при сложенијим захватима.

Образовање – уместо стандардних 2D приказа у уџбеницима, AR омогућава ђацима и студентима интеракцију са 3D моделима (нпр. модели молекула, планета, одређених механичких склопова итд.) што омогућава лакше разумевање тог градива.

Индустрија – техничари на терену могу користити наочаре за проширену стварност или паметне телефоне како би добили инструкције за поправку машина у реалном времену. Инструкције се пројектују директно на делове машине које треба сервисирати и тиме се избегава потреба за обимним приручницима.

Архитектура – архитекте користе ову технологију да би клијентима приказали како ће будући објекат изгледати на празној парцели у стварној величини. Ово омогућава измене дизајна пре самог почетка градње.



Слика 2.4 – Примена проширене стварности у медицини

2.1.3 Негативни утицај и технички изазови проширене стварности

Проширена стварност, иако доноси велике промене у интеракцији са подацима, сама њена реализација се суочава са низом изазова који могу негативно утицати на корисничко искуство. Неки од њих су :

- висока потрошња ресурса и енергије - ове апликације спадају у ред најзахтевнијих софтвера за мобилне уређаје. Истовремено коришћење камере, GPS-а, GPU и сензора за оријентацију доводи до огромног загревања физичке архитектуре и брзог пражњења батерије.
- зависност од спољашњих услова – квалитет директно зависи од осветљења и карактеристика физичког окружења. У условима слабе видљивости алгоритми за праћење простора често изгубе оријентацију што доводи до померања или губитка дигиталних објеката.
- непрецизност локацијских сервиса

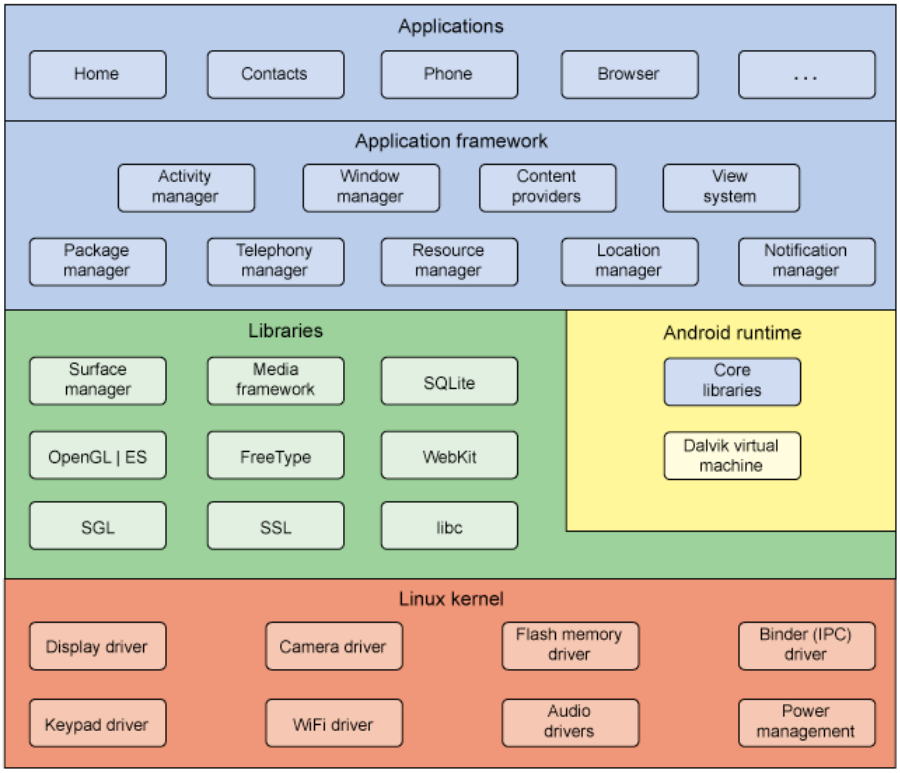
Поред техничких изазова имамо негативан утицај на самог корисника апликације. Један од главних проблема је појава такозване “мучнине изазване симулацијом”. Она настаје услед сензорског конфликта када очи опажају кретање које тело не осећа, што је често последица кашњења у исцртавању слике [11]. Такође, велика количина информација на екрану може преоптеретити корисника и ограничити његову способност брзог доношења одлука.

Постоје и социјални и безбедоносни аспекти. Коришћење камере у јавном простору често покреће питање приватностни других људи. Корисници фокусирани на екран телефона ради праћења AR садржаја могу постати мање свесни свог физичког окружења што представља безбедоносни ризик (у саобраћају, на неравним теренима итд.).

2.2 *Android* оперативни систем

Android представља слојевити софтверски стек отвореног кода заснован на модификованој верзији *Linux* језгра. Платформа је првенствено развијена за мобилне уређаје са екраном осетљивим на додир, али је због своје флексибилности постала водећа платформа за развој решења у областима проширене и виртуелне стварности. Архитектура система је конципирана тако да омогући висок степен апстракције физичке архитектуре, што програмерима дозвољава креирање апликација без одличног познавања специфичности саме физичке компоненте уређаја.

2.2.1 Архитектура *Android* система



Слика 2.5 – Архитектура *Android* оперативног система

Архитектура *Android* оперативног система подељена је у пет кључних слојева, од којих сваки нуди специфичан скуп функционалности за виши ниво [25] :

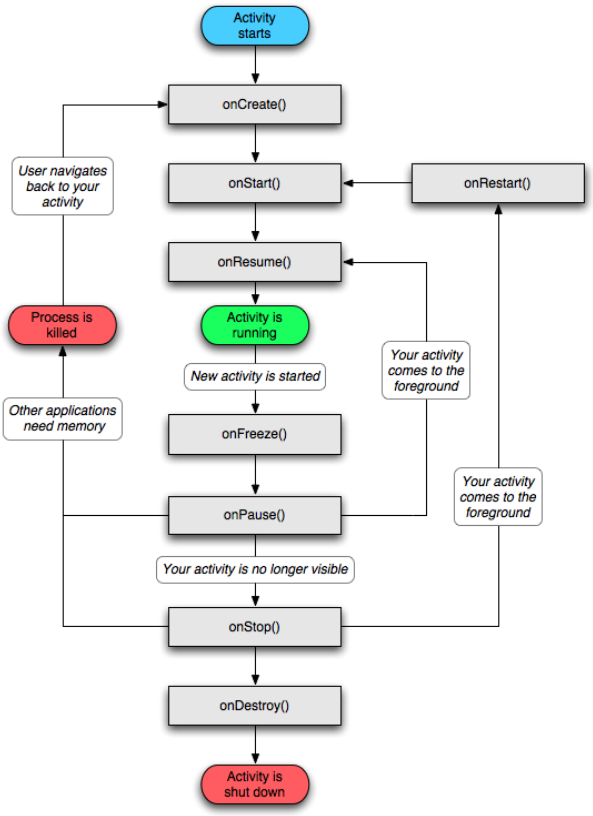
- Језгро оперативног система (енгл. *Linux Kernel*) – Представља темељ целог система. Овај слој је задужен за управљање процесима, меморијом, енергијом као и драјверима за физичку архитектуру као што је камера, *Bluetooth* и GPS. Он делује као слој апстракције између физичке архитектуре и остатка софтверског стека.
- Слој хардверске апстракције (HAL) – Овај слој дефинише стандардне спреге које излажу могућности физичке архитектуре уређаја оквиру апликација (*Java/Kotlin* API). HAL се састоји од више модула библиотека, од којих је сваки реализован за специфичан тип физичке архитектуре.
- *Android Runtime* (ART) – Свака апликација извршава се у сопственом процесу, са својом инстанцом ART – а. Он је дизајниран за извршавање више виртуелних машина на уређајима са ограниченом меморијом, користећи DEX формат датотека за оптимизацију перформанси.
- Апликативни оквир (енгл. *Application framework*) – Представља скуп услуга и класа написаних у *Java* језику које програмери користе за изградњу апликација. Кључне компоненте укључују *Activity Manager* (управља

животним циклусом апликација), *Location Manager* (приступ услугама позиционирања) и *Sensor Manager* (приступ сировим подацима са сензора).

- Апликације (енгл. *Applications*) – Највиши слој који обухвата како системске апликације (контакти, поруке), тако и корисничке апликације као што је ова реализована у овом раду.

2.2.2 Животни циклус активности и управљање ресурсима

Свака апликација на *Android* платформи заснована је на систему активности (енгл. *Activity*), које представљају основне јединице интеракције са корисником. Животни циклус активности (енгл. *Activity Lifecycle*) дефинише низ стања кроз која активност пролази од тренутка покретања до њеног потпуног гашења. Овај систем омогућава оперативном систему да ефикасно управља меморијом и ресурсима, што је од огромног значаја за стабилност уређаја [26].



Слика 2.6 – Животни циклус *Android* апликације

Промене стања контролисане су путем „ *callback* “ метода које систем позива у специфичним тренуцима :

- *onCreate ()* – Ово је прва метода која се позива приликом креирања активности. У овој фази се врши основна иницијализација која треба да се деси само једном током трајања активности, као што су дефинисање корисничког интерфејса,

иницијализација променљивих и повезивање података са елементима екрана. Након извршавања ове методе, активност прелази у стање *Started*.

- *onStart ()* – Метода која чини активност видљивом кориснику. У овој фази се врше финалне припреме за интеракцију. Иако је активност видљива, корисник још увек не може да ступи у интеракцију са њом све док се не изврши следећа метода у низу.
- *onResume ()* – У овој фази активност долази у први план и постаје потпуно интерактивна. Ово је стање у којем се апликација налази већину времена током коришћења. Овде се покрећу све функционалности које захтевају континуирано ажурирање, попут сензорских очитавања или приказа видео преноса.
- *onPause ()* – Ова метода се позива када активност випе није у фокусу (нпр. када се појави системски дијалог). У овом стању активност је и даље делимично видљива, али је неопходно зауставити све операције које троше значајне ресурсе како би систем остао респонзиван.
- *onStop ()* – Апликација није випе видљива кориснику. На пример када је апликација минимизирана, затворена или када друга апликација преузме приоритет. У овом стању апликација задржава своје стање инстанце, али је можда ослободила неке ресурсе.
- *onDestroy ()* – Коначно стање животног циклуса апликације када је потпуно завршена или уклоњена из меморије. То се догађа када систем мора да ослободи ресурсе и одлучи да уништи апликацију. Стање апликације и активне променљиве су изгубљене у овом стању.
- *onRestart ()* – Ако се корисник врати на активност која је била у стању *Stopped*, систем позива ову методу пре поновног покретања активности кроз *onStart ()* фазу.

2.2.3 Сензорски систем и подршка физичке архитектуре

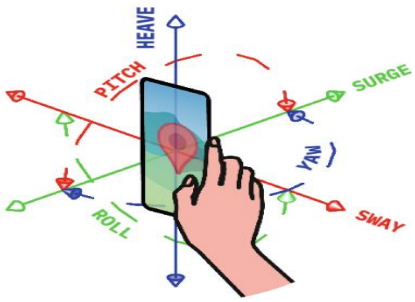
Мобилни уређаји засновани на *Android* платформи опремљени су низом физичких сензора који омогућавају перцепцију физичког окружења. *Android* оквир за сензоре (енгл. *Sensor Framework*) стандардизује начин на који се овим компонентама приступа и како се сирови подаци приказују. Према својој намени, сензори физичке архитектуре се деле на сензоре покрета, сензоре положаја и сензоре околине [27].

У теорији просторног позиционирања, кључну улогу играју следеће компоненте :

- GPS – представља систем за глобалну навигацију који користи податке мреже сателита за израчунавање прецизне географске позиције уређаја. Овај систем омогућава апсолутно позиционирање у глобалном координатном систему, али је ограничен прецизношћу која зависи од доступности сателитског сигнала.

Слика 2.7 – Приказ GPS система

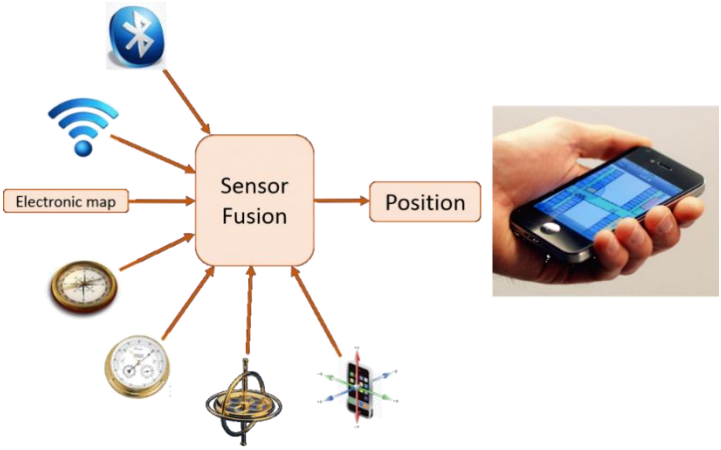
- Акцелерометар – Сензор физичке архитектуре који мери силу убрзања дуж три физичке осе. Поред динамичког убрзања услед померања уређаја, овај сензор мери и статичко убрзање изазвано силом гравитације. Ово омогућава систему да прецизно дефинипе нагиб уређаја у односу на хоризонталну раван.
- Жироскоп – Сензор који мери брзину угаоне ротације уређаја око три физичке осе. Док акцелерометар дефинише нагиб, жироскоп омогућава праћење сложених окрета и оријентације у простору са високим бројем ажурирања. Комбинацијом података са ова два сензора добија се увид у кретање уређаја у 6DoF.



Слика 2.8 – 6DoF – *Six Degrees Of Freedom*

- Магнетометар – Компонента задужена за мерење јачине и правца магнетног поља Земље. У теорији навигације, овај сензор служи као дигитални компас. Омогућава систему да дефинише оријентацију уређаја у односу на магнетне полове Земље, што је кључно за постављање локалних координатних система у односу на стране света.
- Фузија сензора – Представља процес комбиновања података са више физичких сензора како би се смањио шум и неутралисале појединачне мане сваког

сензора. Резултат овог процеса су виртуелни сензори, као што је сензор оријентације, који пружају стабилније и прецизније информације о положају уређаја у стварном свету.



Слика 2.9 – Фузија сензора

2.3 .NET и инфраструктура позадинског система

У архитектури система који омогућавају приказ објеката у проширеној стварности, позадински систем (енгл. *Backend*) представља централни чвор задужен за управљање, складиштење и дистрибуцију података ка мобилним корисницима. Главна улога ове компоненте је обезбеђивање приступа најсвежијим информацијама без потребе за обимним складиштењем на самом уређају. Овакав приступ омогућава централизовано ажурирање података у реалном времену, при чему се све измене ажурирају свим корисницима истовремено.

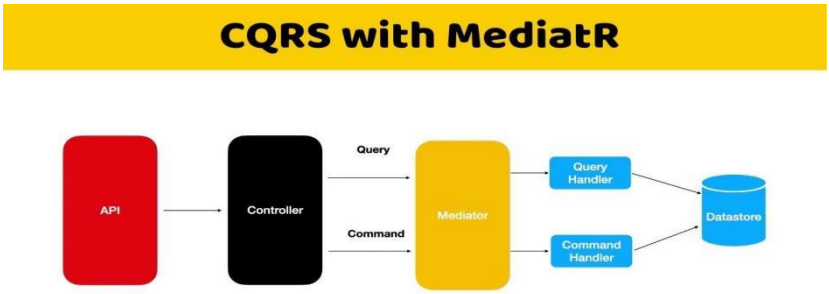
2.3.1 .NET 8 и чиста архитектура

.NET 8 представља једну од најсавременијих верзија *Microsoft* – овог оквира отвореног кода за реализацију модерних апликација високих перформанси [12].

Ради осигурања прегледности и лакшег одржавања софтвера, примењује се концепт чисте архитектуре (енгл. *Clean Architecture*) [13]. Овај концепт организује систем у међусобно независне слојеве :

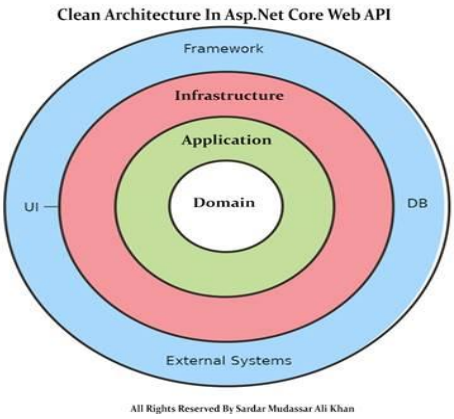
- Доменски слој (енгл. *Domain layer*) – представља срж система и садржи основне ентитете и апстрактне спреге (енгл. *Interfaces*) за складишта података. Дефинише правила рада са подацима независно од конкретних технологија чувања.
- Апликативни слој (енгл. *Application layer*) – дефинише пословну логику и токове података. У овом слоју се често реализује CQRS образац уз помоћ MediatR

библиотеке [14], чиме се постиже раздвајање операција читања података од операција које врше измене података у систему.



Слика 2.10 – Шема CQRS обрасца

- Инфраструктурни слој (енгл. *Infrastructure layer*) – Овај слој је задужен за реализацију приступа спољним ресурсима (базама података, системима датотека) и имплементацију спрега дефинисаних у доменском слоју. Оваква подела омогућава измену специфичних технологија без утицаја на пословну логику система.
- Презентациони слој (енгл. *Presentation layer*) – Представља улазну тачку у систем и задужен је за комуникацију са самом апликацијом. Овај слој излаже функционалности система путем REST приступних тачака. Његова главна улога је прихватање долазних HTTP захтева, њихову валидацију и прослеђивање апликативном слоју ради даље обраде. Након обраде, овај слој је задужен за враћање одговора у JSON формату, уз управљање одговарајућим статусним кодовима, чиме се обезбеђује стандардизована повратна информација кориснику.

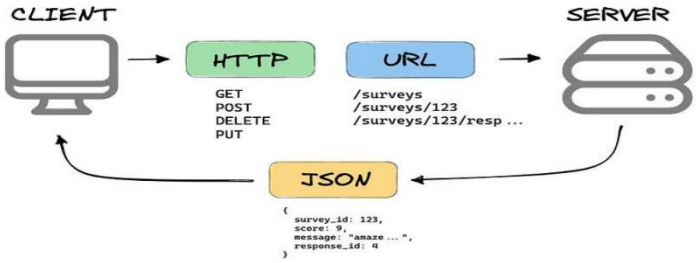


Слика 2.11 – Принцип чисте архитектуре

2.3.2 REST архитектура и размена података

Комуникација између апликације и сервера заснива се на REST архитектури и стандардном HTTP протоколу [15]. Главна карактеристика ове архитектуре је рад без памћења стања (енгл. *statelessness*), што значи да сервер не чува податке о претходним сесијама клијента, већ сваки појединачни захтев садржи све информације за његову обраду. За управљање подацима користе се стандардне HTTP методе :

- GET – за преузимање информација са сервера,
- POST – за креирање нових записа,
- PUT – за ажурирање постојећих података,
- DELETE – за уклањање података из система



Слика 2.12 – Размена података путем REST архитектуре

Размена информација се врши у JSON формату, који због своје структуре омогућава ефикасну серијализацију и десеријализацију објеката на мобилним уређајима, што добро утиче на брзину одзива апликације. За документацију и тестирање API спреге примењује се алат *Swagger*, заснован на *OpenAPI* спецификацији [16]. Овај алат пружа визуелни преглед свих доступних приступних тачака и омогућава директно слање захтева ка серверу ради провере функционалности у реалном времену.

2.3.3 Складиштење података и ORM

За поуздано складиштење података у савременим информационим системима користе се релациони системи (нпр. *Microsoft SQL Server*). Интеракција између програмског кода и базе остварује се путем *Entity Framework Core* алата, који функционише као објектно-релациони мапер (ORM) [17]. Кроз механизам *DbContext* класе, табеле се третирају као објекти, док се упити извршавају путем LINQ језика. Применом *Code-First* приступа и механизма миграција, структура базе се аутоматски усклађује са дефинисаним класама у коду.

У архитектури система који рукују мешовитим садржајем, често се примењује хибридни модел складиштења :

- Релационо складиштење – Користи се за структуиране податке који захтевају брзо претраживање и филтрирање, као што су нумеричке вредности географских координата, називи и категорије објеката
- Складиштење на нивоу датотека – Користи се за неструктуиране садржаје већег обима, попут дугачких текстуалних описа и мултимедијалних датотека.

Овакав модел подразумева да се у релационој бази чувају само референце (нпр. идентификатори), док се стварни садржај повлачи са сервера датотека на захтев корисника, чиме се оптимизују перформансе базе података.

Слика 2.13 – Архитектура хибридног система за управљање подацима кроз .NET Web API и *Entity Framework Core*

2.4 *Google ARCore* платформа

Google ARCore представља софтверски развојни пакет (SDK) који омогућава реализацију напредних искустава проширене стварности на *Android* мобилним платформама. Овај пакет омогућава коришћење AR функционалности без потребе за коришћењем специјализоване физичке архитектуре, попут *LiDAR* сензора, користећи искључиво стандардне компоненте савремених мобилних уређаја: камера, акцелерометар и жироскоп [10]. Ова платформа функционише кроз интеграцију три главна процеса који се извршавају у реалном времену :

- Праћење покрета (енгл. *Motion Tracking*) – Омогућава телефону да разуме своју позицију у односу на физичко окружење.
- Разумевање окружења (енгл. *Environmental Understanding*) – Способност система да детектује карактеристичне тачке и идентификује хоризонталне и вертикалне површине у простору.

- Процена осветљења (енгл. *Light Estimation*) – процена светлости за реалистично сенчење виртуелних објеката.

2.4.1 Визуелно-инерцијална одометрија (VIO)

Основа *ARCore* технологије је процес визуелно-инерцијалне одометрије (VIO) [18]. Систем анализира карактеристичне тачке (енгл. *Feature Points*) у видео запису камере и прати њихову промену положаја током кретања уређаја. Ови визуелни подаци се комбинују са мерењима инерцијалних сензора (акцелерометра и жироскопа) како би се израчунала прецизна локација и оријентација уређаја у простору.

2.4.2 Детекција површина и референтне тачке (*Anchors*)

Процес разумевања окружења омогућава идентификацију геометријских равни у стварном простору (енгл. *Plane Detection*). На основу анализе карактеристичних тачака, алгоритам дефинипе границе хоризонталних и вертикалних површина, као што су тло и зидови објеката.

За прецизно фиксирање виртуелних објеката на одређену локацију у физичком простору користе се референтне тачке (енгл. *Anchor*). Референтна тачка дефинише фиксну тачку у просторном моделу коју систем континуирано математички ажурира [19]. На овај начин се компензују ситне грешке у праћењу и осигурава се да виртуелни објекат задржи свој положај и оријентацију у односу на стварну површину, без обзира на промену позиције корисника.

Слика 2.14 – Детекција површина и приказ карактеристичних тачака

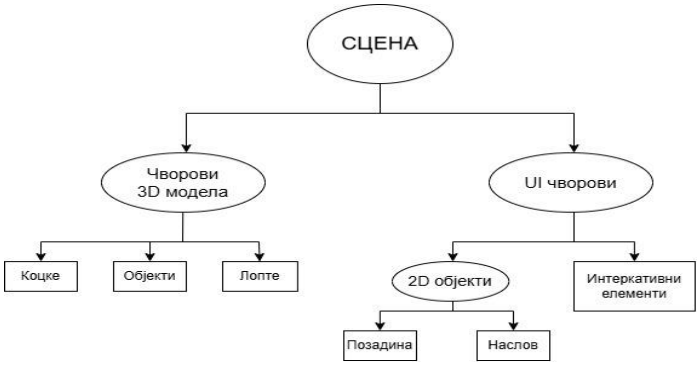
2.5 *Sceneform* библиотека

Sceneform представља софтверску библиотеку намењену за визуелизацију и рендеровање 3D садржаја унутар сцене проширене стварности на *Android* платформи. Она деловање заснива на апстракцији сложених графичких API-ја ниског нивоа (као што је OpenGL), омогућавајући програмерима да се фокусирају на логику сцене уместо на директно управљање графичким ресурсима [20]. Интеграција са *ARCore* платформом омогућава директно везивање дигиталних објеката за тачке у стварном свету.

2.5.1 Хијерархија сцене

За ефикасно управљање виртуелним објектима користи се концепт хијерархије сцене (енгл. *Scene Graph*). У овом моделу, сви елементи унутар сцене проширене стварности организовани су у облику стабла, где сваки елемент представља појединачни чвор (енгл. *Node*). Кључне карактеристике овог концепта су релација родитељ-дете и трансформације.

Релација родитељ-дете подразумева да сваки чвор може имати свој надређени чвор („родитељ“) и подређене чворове („деца“). Свака промена положаја, ротације или размере родитељског чвора директно се рефлектује на све његове подређене чворове.

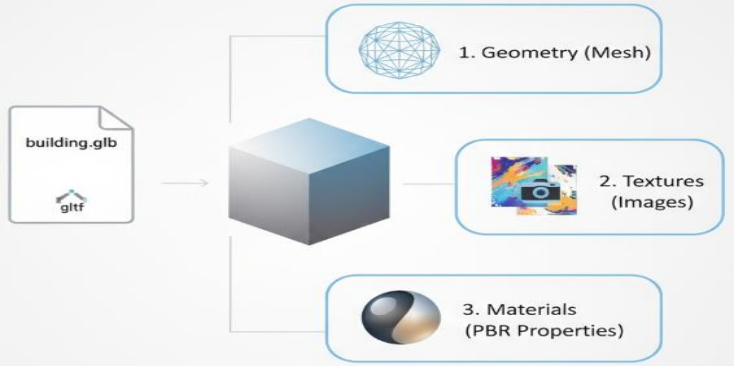


Слика 2.15 – Хијерархијска структура *Sceneform* сцене

Трансформације омогућавају да се групе објеката померају и мењају као јединствена целина, чиме се значајно поједностављује математички прорачун координата и олакшава управљање међусобним односима елемената унутар 3D простора.

2.5.2 GLB формат 3D модела и PBR рендеровање

GLB формат представља индустријски стандард за ефикасан пренос и учитавање 3D сцена и модела у апликацијама проширене стварности. Овај формат је заправо бинарна верзија glTF стандарда, оптимизована тако да у оквиру једне датотеке обједињује геометрију, текстуре и дефиниције материјала [21]. Оваква структура значајно убрзава процес рендеровања и смањује број мрежних захтева, што је од велике важности за стабилан рад на мобилним уређајима са ограниченим ресурсима физичке архитектуре.



Слика 2.16 – Структура једног GLB фајла

Савремени системи за графички приказ у оквиру *Sceneform* библиотеке заснивају се на PBR принципима. Коришћењем ове методе материјали виртуелних објеката дефинишу се према физичким својствима стварних површина што омогућава реалистично осветљење као и ефикасност графичког преноса.

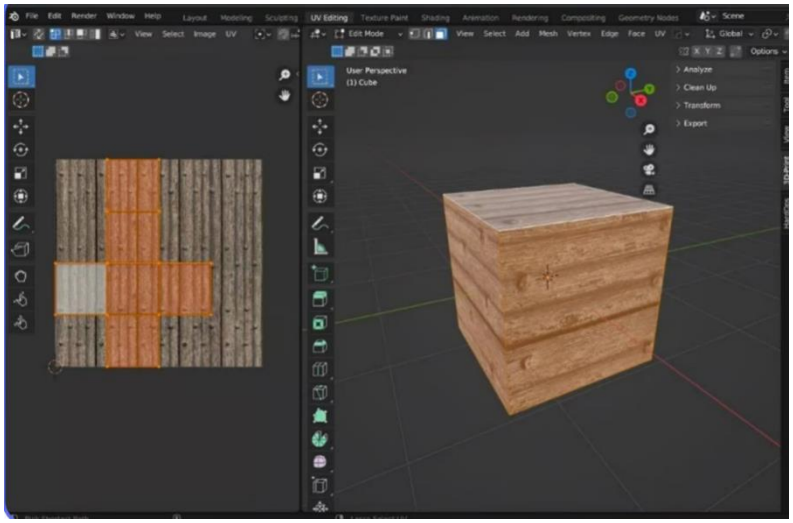
Реалистично осветљење подразумева да систем користи податке о осветљењу самог окружења које прикупља *ARCore* како би динамички прилагодио сенчење и рефлексије, постижући висок степен визуелне усклађености између дигиталног и физичког света.

Ефикасност графичког преноса се односи на то да директан пренос бинарних података ка GPU смањује кашњење и потрошњу системске меморије, чиме се осигурава стабилан рад апликације у реалном времену.

2.5.3 UV мапирање текстура

UV мапирање представља стандардни процес пројектовања 2D слике на површину 3D објекта. Назив потиче од оса 2D простора текстуре (*U* за хоризонталну и *V* за вертикалну осу), које се користе како би се избегла забуна са *X, Y, Z* осама у 3D простору.

Главни корак у овом процесу је UV расклапање мреже (енгл. *unwrapping*) где се геометрија 3D објекта виртуелно пресеца и прелази у 2D раван. Ово омогућава графичком покретачу да прецизно повеже сваки пиксел слике са одговарајућом тачком на 3D моделу и тиме осигуравамо правилан приказ материјала без деформација.



Слика 2.17 – UV мапирање у алату *Blender*

2.5.4 Визуелна стабилност и глаткоћа приказа

У рачунарској графици у реалном времену, један од главних изазова је одржавање глатког приказа при великом броју кадрова у секунди (FPS). Како би се избегло блокирање главне извршне нити апликације, учитавање ресурса (3D модела и текстура) врши се

асинхроно. Додатно, у циљу елиминације визуелног „подрхтавања“ објеката услед варијације података са сензора, примењују се математичке методе за глатко прелажење између стања, чиме се виртуелни објекти фиксирају за стварни свет на природнији начин.

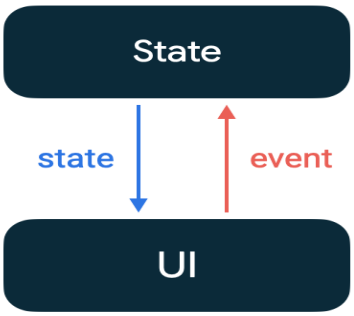
2.6 *Jetpack Compose* и декларативни приступ развоју интерфејса

У савременом развоју *Android* апликација, *Jetpack Compose* представља модеран софтверски оквир за реализацију изворног корисничког интерфејса [22]. За разлику од старог императивног приступа који се ослања на XML датотеке и ручно ажурирање стања појединачних елемената, *Jetpack Compose* уводи декларативни приступ.

2.6.1 Декларативни приступ и управљање стањима

Суштина декларативног приступа је у дефинисању интерфејса као функције тренутног стања података. Кориснички интерфејс се описује кроз композитне функције (енгл. *Composable functions*) означене анотацијом *@Composable*. Ове функције примају податке као параметре и аутоматски обнављају делове приказа када дође до промене стања, што је процес познат као рекомпозиција. На овај начин се постиже већа стабилност кода и лакше одржавање, јер систем сам брине о усклађености података и њиховог визуелног приказа.

Овај приступ функционише кроз механизам једносмерног протока података, где разликујемо два кључна елемента: стање (енгл. *State*) и догађај (енгл. *Event*). Стање представља податке који описују тренутну ситуацију у апликацији. Промена стања аутоматски покреће освежавање интерфејса. Догађај представља акције које долазе са корисничког интерфејса и шаљу се логици апликације како би се, по потреби, ажурирало стање.



Слика 2.18 – Механизам једносмерног протока података *Jetpack Compose*

2.6.2 Компатибилност са постојећим системима

Иако *Jetpack Compose* у потпуности мења начин реализације корисничког интерфејса, он је дизајниран да буде компатибилан са постојећим *Android* компонентама. Ово омогућава коришћење специјализованих погледа који још увек нису део *Compose* окружења кроз концепт *AndroidView* адаптера. На овај начин се постиже спој модерног управљања стањем и напредних графичких могућности које пружа *ARCore*.

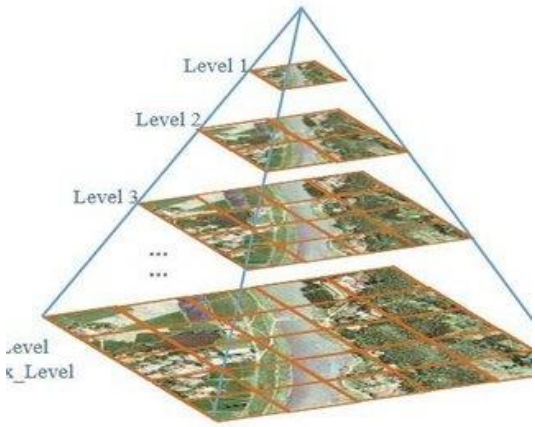
2.7 Системи за приказ дигиталних мапа

У системима који захтевају интеграцију географских података са мобилним платформама, често се јавља потреба за поузданим и слободно доступним решењима за 2D приказ картографских основа. Библиотека *Osmdroid* представља решење овог типа, служећи као софтверски оквир који омогућава приказ мапа заснованих на *OpenStreetMap* подацима унутар *Android* окружења [23].

2.7.1 *OpenStreetMap* и плочасти приказ

OpenStreetMap представља глобални пројекат слободних географских података који се заснива на принципима заједничког доприноса корисника [24]. За разлику од векторских модела, већина мобилних реализација које користе овај извор ослања се на плочасти приказ (енгл. *Map Tiles*).

Овај концепт подразумева поделу мапе на квадратне графичке сегменте (плочице) унапред утврђених димензија. Ови сегменти се хијерархијски организују према нивоима увеличавања, што омогућава систему да динамички преузима и приказује само оне делове мапе који су тренутно значајни за корисниково видно поље, чиме се оптимизује потрошња меморијских ресурса.



Слика 2.19 – Хијерархијска структура плочица унутар *OpenStreetMap* система

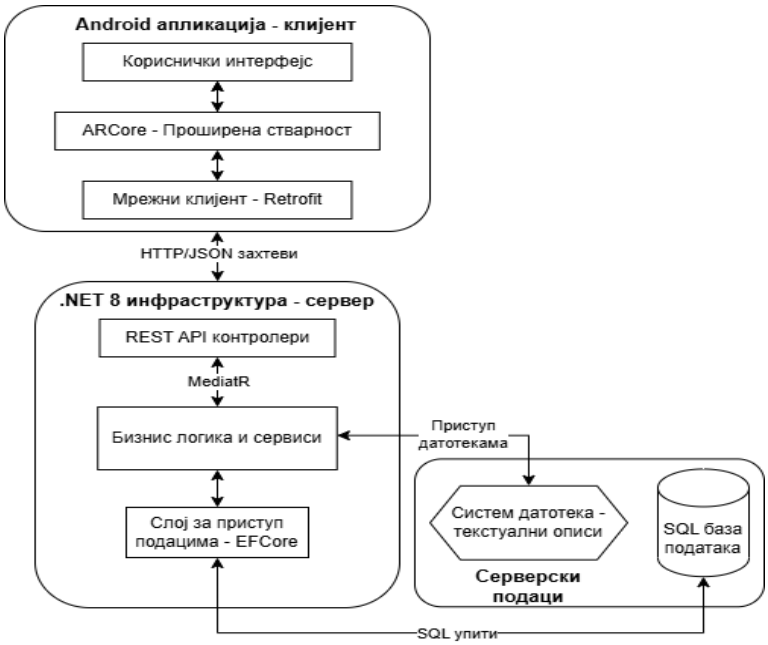
2.7.2 Карактеристике и функционалности библиотеке

Библиотека *Osmdroid* омогућава слојевиту организацију података, што је од велике важности за комплексност приказа географских информација. Кључне карактеристике овог софтверског решења су :

- Механизам кеширања – Сваки учитани сегмент мапе привремено се складишти у локалној меморији уређаја. Овај механизам омогућава брз приступ подацима приликом поновног прегледа истог подручја, смањујући мрежни саобраћај и побољшавајући респонзивност приказа.
- Слојевити приказ – Софтверска архитектура подржава исцртавање додатних информационих слојева преко основне мапе. Ови слојеви омогућавају приказ тачака од интереса, геометријских облика или индикатора тренутне позиције, без промене подлоге мапе.
- Вишенитно процесирање – Процес преузимања и рендеровања мапа одвија се асинхроно, чиме се обезбеђује континуитет у раду корисничког интерфејса чак и приликом честих промена географских координата.

3. Концепт решења

У овом поглављу биће описан концепт програмског решења апликације која је реализована коришћењем *Android Studio* развојног окружења и намењена је покретању на мобилним уређајима са *Android* оперативним системом и подршком за проширену стварност. Решење можемо поделити на две главне целине. Прву чини *Android* мобилна апликација, развијена у језику *Kotlin*, која се користи за креирање и дефинисање изгледа и функционалности, укључујући интеграцију мапа, логику проширене стварности и интеракцију са корисником. Друга целина представља позадинску инфраструктуру засновану на .NET 8 оквиру, која обухвата припрему података, њихово складиштење у бази и дистрибуцију ка мобилном уређају путем веб сервиса. На слици 3.1 приказана је архитектура апликације која обухвата кључне компоненте одговорне за њену функционалност.



Слика 3.1 – Дијаграм модула који чине решење

За почетак развоја апликације неопходно је подесити два развојна окружења на рачунару. Прво окружење је *Visual Studio*, у којем се реализује серверска логика и база података, омогућавајући динамичко управљање информацијама о објектима. Друго окружење чини *Android Studio*, који омогућава креирање и тестирање пројекта пре него што се он припреми за извршавање на крајњем уређају. Да би се од пројекта развијеног у *Android Studio* – у добила нативна *Android* апликација, потребно је следити кораке који укључују подешавање *Android* SDK – а и NDK – а, као и конфигурацију специфичних модула попут *ArCore* – а за рад са проширеном стварношћу.

Главни алат који се користи за претварање развојне верзије пројекта у нативну *Android* апликацију је *Gradle build* систем. Да би апликација правилно функционисала, потребно је у подешавањима пројекта дефинисати минималну верзију система и неопходне захтеве физичке архитектуре за камеру и сензоре. Након завршене реализације у развојном окружењу, пројекат се преводи за *Android* платформу. Добијена APK датотека може се инсталирати на уређај директно или путем ADB алата.

ADB омогућава детаљну анализу рада система у реалном времену, што је од великог значаја за овај рад. Путем овог алата могуће је пратити перформансе уређаја приликом рендеровања AR садржаја, вршити дебаговање мрежне комуникације са .NET сервером, као и пратити стање меморије и употребу процесорских језгара. Иако се за развој може користити емулатор, за проверу стварних перформанси проширене стварности и прецизности GPS лоцирања, неопходно је покренути апликацију као нативну верзију на реалном *Android* мобилном уређају.

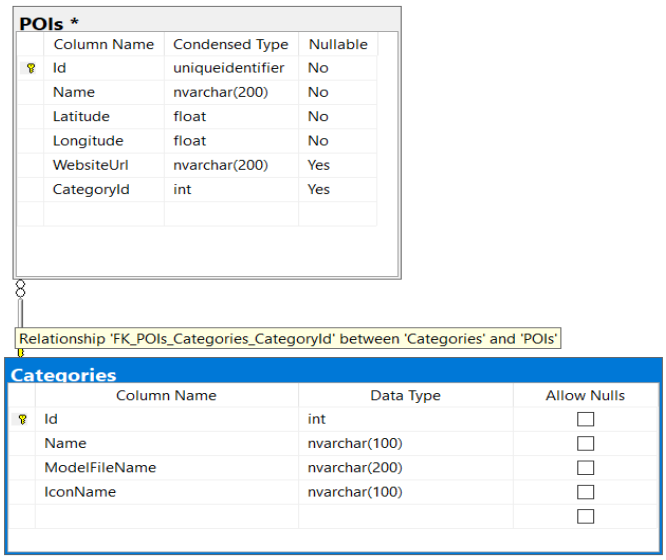
3.1 Логичка организација и структура података

Како би систем функционисао ефикасно у реалном времену, концепт решења предвиђа специфичну хибридну организацију података. Овакав приступ је неопходан како би се постигао баланс између перформанси и количине информација које се приказују кориснику. Сви подаци потребни за рад система складиштени су на серверској страни и логички су подељени у две основне целине.

Прву категорију података на серверу чине релациони геопросторни подаци смештени у централној SQL бази података. Овај сегмент обухвата лагане, али битне информације неопходне за математичке прорачуне и почетно рендеровање у проширеној стварности. Сваки објекат у систему је представљен кроз следећи скуп атрибута :

- Идентификатор (ID) – Јединствена вредност која омогућава прецизно мапирање између базе и визуелног маркера на екрану.

- Географске координате – Чувају се у формату високе прецизности. Свака децимала је битна јер директно утиче на то да ли ће маркер у AR приказу бити тачно постављен на објекат или ће бити померен у простору
- Назив и категорија – Кратке информације које се приказују у примарном слоју. Категорије су концептуално одвојене и повезане релацијом са тачкама интереса преко спољног кључа.

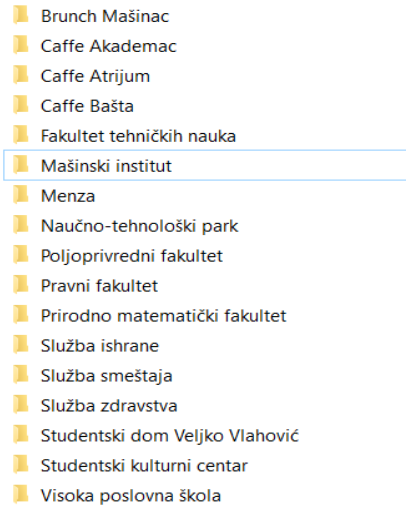


Слика 3.2 – Шема релационе базе података са дефинисаним POI објектима и категоријама

Оваква организација, као што је приказано на слици 3.2, омогућава лако филтрирање објеката на екрану према типу, чиме се кориснику пружа персонализовано искуство и спречава претрпавање интерфејса маркерима.

Другу целину представља серверски систем датотека. За разлику од основних атрибута, детаљни описи објеката третирају се као засебни ресурси. Концепт предвиђа коришћење текстуалних датотека смештених на серверу из следећих разлога:

- Оптимизација мрежног саобраћаја – Апликација не преузима тешке текстуалне описе приликом почетне синхронизације, већ само основне координате. Тек на кориснички захтев, систем повлачи одређену датотеку.
- Флексибилност ажурирања – Администратор може брзо изменити опис у датотеци без потребе за изменом структуре саме базе података.



Слика 3.3 – Аутоматски креирани директоријуми за опис тачака од интереса

3.2 Опис апликације и корисничког интерфејса

Концепт корисничког интерфејса ове апликације није замишљен само као визуелни оквир, већ као функционални мост између физичког простора и дигиталних података. Главни циљ дизајна је минимализам, како би се омогућило кориснику да се фокусира на информације о проширеној стварности, без сувишних елемената који би ометали преглед околине кроз камеру.

Апликација је разрађена као систем који интегрише три кључна аспекта интеракције :

1. Визуелизација (преглед објеката у реалном простору)
2. Оријентација (сналажење помоћу дводимензионалне мапе)
3. Администрација (управљање подацима на терену)

Корисничко искуство је реализовано тако да буде интуитивно, омогућавајући брз прелазак између модула у зависности од начина употребе. На пример, корисник може користити мапу да види где је удаљени објекат, а после тога прећи на AR приказ за крајњу идентификацију и приказ детаљних информација када се нађе у непосредној близини.

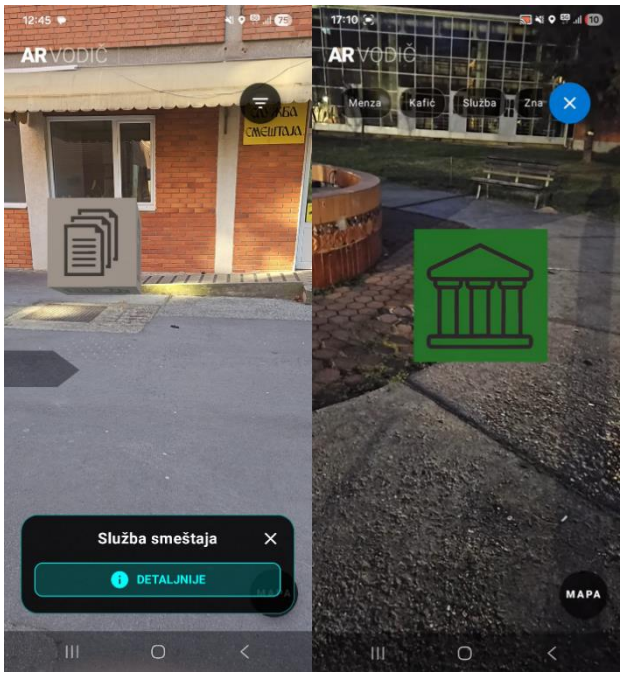
Оваква структура омогућава да апликација буде истовремено и алат за навигацију и ефикасан систем за прикупљање података у реалном времену, што ће бити детаљније описано у наредним поглављима.

3.2.1 Модул проширене стварности

Модул проширене стварности представља примарни интерфејс апликације чији је задатак да дигиталне информације из базе података просторно постави у физички простор корисника. За разлику од стандардних 2D приказа, овај модул користи видео пренос са камере уређаја као позадински слој на којем се рендерују виртуелни објекти.

Кључне компоненте овог приказа су :

- Динамички AR маркери – Свака локација је представљена виртуелним објектом који на додир кориснику пружа најбитније информације (назив и удаљеност) у реалном времену. Концепт предвиђа да се маркери прилагођавају положају уређаја, обезбеђујући прецизну идентификацију објеката у три димензије.
- Селективни приказ садржаја – Како би се избегла визуелна претрпаност екрана, кориснику је омогућено да по потреби активира мени за филтрирање. Овај систем дозвољава филтрирање објеката по категоријама, чиме се у реалном времену смањује број приказаних маркера само на оне из категорије коју је корисник изабрао.
- Слојевитост детаља – Интерфејс је дизајниран да не преоптерећује корисника текстом. Информације су подељене у слојеве, од основних података које корисник добије притиском на маркер, до детаљних описа који се позивају само на захтев корисника.



Слика 3.4 – Кориснички интерфејс AR модула

3.2.2 Модул интерактивне мапе

Док је модул проширене стварности фокусиран не непосредно препознавање објеката у видном пољу, део са интерактивном мапом замишљен је као алат за оријентацију и сналажење у ширем простору кампуса. Концепт овог решења се разликује од уобичајених навигационих апликација по томе што не захтева потпуно напуштање AR сцене. Уместо

тога, реализован је хибридни, односно интерфејс са подељеним екраном (енгл. *Split-screen*), где мапа заузима доњи део екрана, док у горњем делу и даље остаје активан видео пренос са камере (слика 3.5). Оваква организација простора на екрану омогућава кориснику да истовремено користи предности оба модула – да прати своју позицију на 2D мапи и истовремено посматра стварно окружење кроз објектив уређаја.

Важан део овог модула је непрекидно праћење локације корисника, која је на мапи представљена специфичним маркером. Систем у реалном времену ажурира овај приказ на основу података са GPS сензора, омогућавајући кориснику да у сваком тренутку тачно зна где се налази у односу на околне зграде.

Додатни ниво интеракције постигнут је кроз могућност директног приступа информацијама путем мапе. Интеракцијом са било којим маркером на мапи кориснику се приказује назив локације и прецизна удаљеност у односу на његову тренутну позицију. Ова функционалност осигурава усклађеност информација унутар целе апликације, јер се исти подаци који су доступни у AR приказу могу лако проверити и на мапи. Такође, концепт филтрирања омогућава да свака промена категорије у главном менију истовремено ажурира и приказ на мапи.

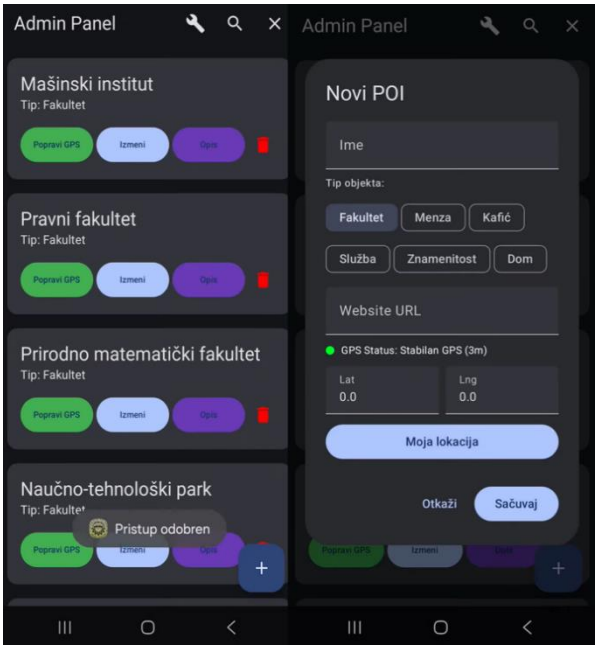
Слика 3.5 – Приказ интерактивне мапе

3.2.3 Административни модул

Административни модул замишљен је као главни интерфејс за управљање геопросторним подацима и садржајем апликације. За разлику од корисничког дела који је фокусиран на визуелно искуство, овај модул омогућава овлашћеним корисницима потпуну контролу над базом података кроз операције креирања, ажурирања и брисања објеката. Концепт је заснован на прегледности и брзини уноса, где је свака локација представљена у виду картице са основним подацима, док реализован систем претраге омогућава администратору да тренутно пронађе жељени објекат према називу и категорији.

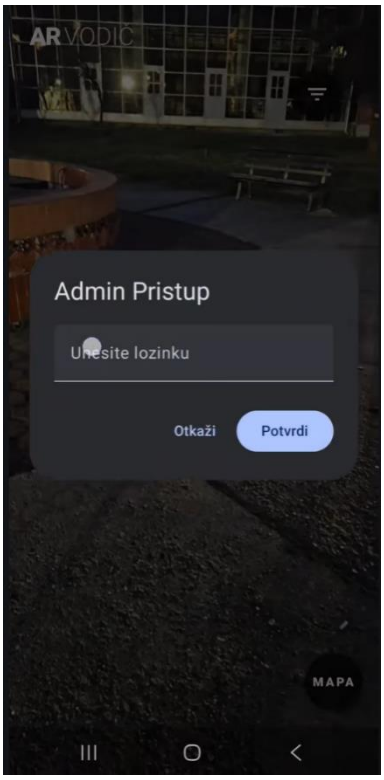
Посебан фокус у изради овог модула стављен је на прецизност прикупљања локацијских података. Администратору је на располагању више начина дефинисања координата: путем ручног уноса географске дужине и ширине, што омогућава велику прецизност али може довести до грешке при куцању, или коришћењем наменске функције за аутоматско преузимање тренутне позиције уређаја. Ова друга опција је од великог значаја за рад на терену, јер омогућава администратору да стане испред стварног објекта и притиском на тастер усклађује дигиталне координате са физичком локацијом, чиме се смањује могућност раније поменуте грешке при ручном уносу.

Поред позиционирања, модул омогућава и управљање детаљним описима који се на сервер шаљу путем REST API-ја. Концепт предвиђа да се свака измена, било да је реч о текстуалном опису или промени локације, одмах пропагира кроз систем. Након успешне комуникације са сервером, апликација аутоматски освежава податке у свим осталим модулима (AR приказ и мапа), чиме се осигурава да су информације које корисници виде на уређајима увек актуелне и тачне.



Слика 3.6 – Изглед админ панела и форме за додавање нове локације

Заштита овог модула заснована је на систему двоструке провере, чији је циљ да онемогући случајни или неовлашћени приступ критичним подацима. Први ниво заштите ослања се на принцип прикривености, где се приступ за администрацију активира специфичном секвенцом интеракција која није видљива нити позната обичном кориснику. Тек након тога, систем захтева унос сигурносне лозинке.



Слика 3.7 – Дијалог за унос лозинке и приступ админ панелу

4. Програмско решење

У овом поглављу детаљно је приказан поступак реализације претходно дефинисаног концепта апликације за приказ тачака од интереса у проширеној стварности. Фокус је стављен на техничку реализацију кључних модула, конфигурацију развојног окружења, као и на алгоритме који омогућавају прецизно позиционирање дигиталних објеката у стварном простору. Поред клијентског дела, поглавље обухвата и реализацију позадинског система заснованог на .NET 8 технологији, моделовање базе података, као и успостављање сигурне везе између сервера и мобилног уређаја. Такође, поглавље обухвата опис административног панела и механизама безбедности који осигуравају саме податке из базе података.

4.1 Развојно окружење и конфигурација

За развој мобилне апликације коришћено је окружење *Android Studio*, које представља званични алат за израду *Android* софтвера. Овај IDE је изабран због његове подршке за језик *Kotlin,* интегрисаног менаџера спољних библиотека кроз *Gradle* систем, као и због одличних алата за дебаговање апликација које користе проширену стварност. Посебан значај имао је *Logcat* алат за праћење рада сензора у реалном времену.

Апликација је конфигурисана тако да захтева минималну верзију оперативног система *Android 8.0* (API Level 26), што је доњи праг за стабилан рад *ARCore* платформе. С друге стране, циљна верзија постављена је на API Level 36. Ово омогућава апликацији да користи најнапредније оптимизације система, нарочито у погледу приватности података о локацији и ефикаснијег управљања енергијом приликом коришћења камере у AR режиму.

4.1.1 Подешавање пројекта и *Gradle* зависности

Пројекат је организован помоћу *Gradle* алата за аутоматизацију изградње софтвера. Све спољне библиотеке дефинисане су унутар *build.gradle.kts* датотеке. Овакав приступ

омогућава лаку контролу верзија и аутоматско преузимање неопходних компоненти са централних репозиторијума [28]. У наставку је приказан кључни део из конфигурационе датотеке са дефинисаним верзијама SDK – а и главним зависностима

Део датотеке build.gradle.kts :

```
android {
        compileSdk {
            version = release(36)
          }
        defaultConfig {
            applicationId = "com.example.projekat"
            minSdk = 26
            targetSdk = 36
            versionCode = 1
            versionName = "1.0"
          }
}
dependencies {
        //Jetpack Compose
        implementation(platform(libs.androidx.compose.bom))
        implementation(libs.androidx.compose.ui)
        //ARCore i Sceneform
        implementation("com.google.ar:core:1.51.0")
        implementation("com.google.ar.sceneform:core:1.17.1")
        implementation("com.google.ar.sceneform.ux:sceneform-ux:1.17.1")
        //Mape i veza
        implementation("org.osmdroid:osmdroid-android:6.1.18")
        implementation("com.squareup.retrofit2:retrofit:2.9.0")
        implementation("com.squareup.retrofit2:converter-gson:2.9.0")
}
```

4.1.2 Интеграција *ARCore* и *Sceneform* библиотеке

Техничка реализација AR модула започета је интеграцијом *Sceneform* библиотеке, која омогућава везивање 3D модела за *ARCore* референтне тачке. Кључни корак у конфигурацији је постављање апликације као „ *AR Required* “, што се постиже изменама у манифесту. За разлику од теоријских поставки поменутих раније, реализација захтева специфично руковање дозволама у реалном времену.

У датотеци *AndroidManifest.xml* дефинисани су захтеви физичке архитектуре:

```
<uses-feature android:name = "android.hardware.camera.ar" android:required = "true"
/>
<application …
    <meta-data
        android:name = "com.google.ar.core"
        android:value = "required" />
```

С обзиром на то да *Sceneform* захтева одређене OpenGL верзије, у *build.gradle.kts* датотеци је додатно конфигурисана подршка за *Java 11* опције, што је технички предуслов за исправно превођење библиотеке :

```
android{
        compileOptions {
            sourceCompatibility = JavaVersion.VERSION_11
            targetCompatibility = JavaVersion.VERSION_11
      }
        kotlinOptions {
            jvmTarget = "11"
      }
}
```

4.1.3 Конфигурација *Osmdroid* библиотеке за рад са мапама

За израду интерактивне 2D мапе коришћена је библиотека *Osmdroid*. За разлику од комерцијалних решења, ова библиотека захтева ручну конфигурацију система за складиштење података и специфично управљање мрежним захтевима. Основна предност ове конфигурације је могућност рада са слободним изворима мапа и потпуна контрола над кеширањем сегмената мапе (енгл. *Tiles*)

Први корак у конфигурацији је дефинисање дозвола унутар *AndroidManifest.xml* датотеке које омогућавају апликацији преузимање географских података у реалном времену и прецизно позиционирање корисника на мапи:

```
<uses-permission android:name = "android.permission.INTERNET" />
<uses-permission android:name = "android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name = "android.permission.ACCESS_FINE_LOCATION" />
```

Поред дозвола, реализација захтева специфичну иницијализацију унутар главног програмског кода. Главни изазов код *Osmdroid* – а је препознавање апликације ка серверима са мапама, што се решава постављањем *userAgentValue* параметра. Како је циљна верзија система API 36, апликација користи интерно складиште за кеширање мапа, чиме се осигурава брз одзив интерфејса без потребе за додатним дозволама за складиштење.

Иницијализација конфигурације:

```
Configuration.getInstance().userAgentValue = context.packageName
```

Овим је завршена основна конфигурација, чиме је омогућено да мапа исправно учитава податке, док се даља подешавања изгледа и интерактивности врше директно приликом креирања корисничког интерфејса.

4.2 Реализација корисничког интерфејса

Интерфејс апликације реализован је коришћењем *Jetpack Compose* оквира, при чему је примарни фокус стављен на вишеслојну архитектуру приказа. Како је AR сцена динамична и захтевна по ресурсе, интерфејс мора да буде реактиван – да се појављује само када је то неопходно. За постизање ове функционалности, главну улогу имају стања која дефинишу тренутни изглед екрана. У основном *Composable* модулу, ова стања су дефинисана помоћу *remember* и *mutableStateOf* функција. Оне омогућавају апликацији да задржи вредности променљивих током рекомпозиције, што је кључно за несметан рад AR сцене и корисничког интерфејса:

```
//Дефинисање кључних стања за управљање приказом
var isVertical by remember{mutableStateOf(false)}
var isInitializing by remember{mutableStateOf(True)}
var errorMessage by remember{mutableStateOf<String?>(null)}
var selectedPoi by remember(mutableStateOf<PoiDto?>(null))
```

4.2.1 Развој компоненти и вишеслојна архитектура приказа

Изградња корисничког интерфејса заснована је на структури независних и поново употребљивих компоненти. С обзиром на то да је фокус апликације примарно на проширеној стварности, основна структура је организована помоћу *Box* распореда који омогућава преклапање елемената по Z – оси. Први слој заузима *ARSceneView,* док се изнад њега исцртавају информативни и контролни слојеви:

```
Box(modifier = modifier.fillMaxSize()){
    //Основни слој - рендеровање AR сцене преко целог екрана
    AndroidView(
        factory = {arSceneView},
        modifier = Modifier.fillMaxSize()
      )
      //Упозорење за вертикални положај
      if(!isInitializing && !isVertical){
        VerticalPhoneOverlay()
      }
      //Кориснички интерфејс и контроле
      Column(
        modifier = Modifier
            .fillMaxWidth()
            .StatusBarsPadding()
            .padding(top = 16.dp, start = 20.dp, end= 20.dp)
      ){
        //Део где се налазе лого, админ приступ, мапа
      }
}
```

Овакав распоред осигурава да AR приказ заузима целокупну површину екрана, док се елементи попут *VerticalPhoneOverlay* појављују само као реакција на промену стања *isVertical.* Тиме се постиже оптимизација у коришћењу ресурса јер *Jetpack Compose* поново исцртава само оне слојеве чије стање је промењено.

4.2.2 Динамичко прилагођавање интерфејса и системска обавештења

Уместо статичног интерфејса, апликација користи систем који се прилагођава тренутној намери корисника и условима на терену. Овај процес почиње на нивоу селекције садржаја, односно филтрирања категорија где интерактивна трака за филтрирање омогућава кориснику да одабере која категорија објеката ће се приказивати и тиме побољшава прегледност саме сцене.

Упоредо са филтрирањем садржаја, апликација надгледа физичку стабилност приказа. У ситуацијама када сензори детектују неправилан нагиб, активира се заштитни слој за вертикални положај уређаја. Овај слој делује тако што привремено прекрива сцену, давајућу кориснику до знања да треба уређај да врати у одређени положај без потпуног прекида рада апликације.

Сличан принцип примењен је и на систем обавештења. Када дође до техничких потешкоћа, попут губитка интернета или грешке приликом очитавања базе података, апликација користи картице упозорења на дну екрана. Ово омогућава кориснику да и даље прати окружење, док истовремено добија прецизан опис грешке кроз дефинисана стања:

```
//Приказ грешке на основу стања
if(errorMessage != null){
    Card(
        modifier = Modifier.fillMaxWidth().padding(16.dp),
        colors = CardDefaults.cardColors(containerColor =
                    Color.red.copy(alpha = 0.8))
       ){
        Row(modifier = Modifier.padding(12.dp),
        verticalAlignment = Alignment.CenterVertically
        ){
            Icon(Icons.Default.Warning, contentDescription = null,
                tint = Color.White)
            Text(
                text = errorMessage!!
                modifier = Modifier..weight(1f).padding(horizontal = 8.dp)
            )
            TextButton(onClick = {refreshPois()}){
                Text("POKUŠAJ PONOVO",color = Color.White)
            }
        }
       )
```

Последњи ниво интеракције остварује се кроз информативне панеле, који се активирају тек након директног одабира одређеног маркера у простору. Селектована тачка интереса тада отвара додатни слој са описом и прецизном удаљеношћу, чиме се заокружује цео процес од претраживања до детаљног информисања унутар јединственог корисничког интерфејса.

4.2.3 Интерфејс навигационог слоја и интерактивне мапе

Како би се кориснику олакшала оријентација у ширем окружењу, AR приказ је допуњен навигационим слојем у виду интерактивне мапе која функционише као слој изнад примарне сцене. Мапа је дизајнирана да не заклања цео видокруг камере, а активира се искључиво на захтев корисника путем плутајућег дугмета (енгл. *Floating Action Button*). Овај приступ омогућава несметано истраживање терена уз брз приступ гегорафском прегледу локација.

Технички сегмент овог слоја заснива се на усклађивању података са AR сценом. Коришћењем стања *selectedType*, листа објеката на мапи се аутоматски филтрира, осигуравајући да корисник види идентичне податке у оба режима приказа.

```
// 1. Логика филтрирања
val filteredPois = remember(poiList, selectedType) {
    if (selectedType == "Sve") poiList
    else poiList.filter { it.categoryName == selectedType }
}
if (isMapExpanded) {
    Box(modifier = Modifier.fillMaxSize().pointerInput(Unit) {
        detectTapGestures(onTap = { isMapExpanded = false })
    })
}
// 3. Контејнер са анимацијом промене стања (Мапа <-> Дугме)
Box(modifier = Modifier.align(Alignment.BottomEnd).padding(16.dp)) {
    AnimatedContent(targetState = isMapVisible) { visible ->
        if (visible) {
            Card(modifier = Modifier.size(width = mapWidth, height = mapHeight)) {
                Box(modifier = Modifier.fillMaxSize()) {
                    MiniMapOSM(currentLocation = location, poiList = filteredPois)
                    // Дугмад заFullscreen и затварање
                    IconButton(onClick = {isMapExpanded = !isMapExpanded}){/*Icon*/}
                    IconButton(onClick = { isMapVisible = false }){/*Icon*/}
                }
            }
        } else {
            FloatingActionButton(onClick = { isMapVisible = true }) {
                Text("МАРА")
            }}}}
```

За транзицију између приказа користи се *AnimatedContent* који реагује на промену стања *isMapVisible*, претварајући плутајуће дугме у функционални прозор мапе. Додатни ниво интеракције омогућен је кроз стање *isMapExpanded*, које контролише димензије картице. Како би се поједноставила употреба, реализован је посебан *Box* слој који, када је мапа проширена, детектује додир било где изван њених оквира и аутоматски ресетује приказ.

4.2.4 Административни интерфејс за управљање подацима

За овлашћене кориснике израђен је администартивни модул који омогућава потпуну контролу над тачкама интереса директно кроз мобилну апликацију. Овај део интерфејса реализован је као засебан екран заснован на *Scaffold* архитектури, која интегрише динамичку претрагу унутар навигационе траке.

Централни део интерфејса чини динамичка листа (*LazyColumn*), где свака тачка заузима засебну картицу са јасно дефинисаним акцијама. Дизајн је фокусиран на ефикасност, па су најчешће операције, попут калибрације GPS, доступне само једним кликом. Дугме за то је издвојено и омогућава тренутно усклађивање координата објекта са тренутном локацијом уређаја, чиме се избегава мануелно уношење и смањује могућност грешке.

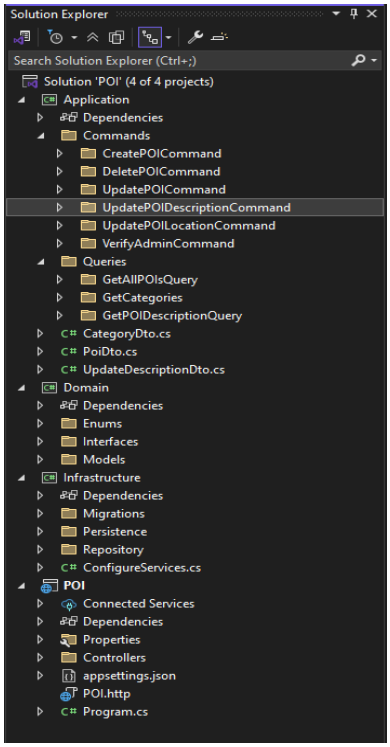
```
//Приказ појединачне локације са брзим акцијама у админ панелу
items(filteredPois, key = {it.id}){poi ->
    Card(
        modifier = Modifier.fillMaxWidth().padding(8.dp)
    ){
        Column(modifier = Modifier.padding(16.dp)){
            Text(poi.name, style = MaterialTheme.typography.titleLarge)
            Row(
                modifier = Modifier.fillMaxWidth().padding(top = 12.dp),
                horizontalArrangement = Arrangement.spacedBy(4.dp)
            ){
                //Дугме за калибрацију локације на терену
                Button(onClick = { currentLocation?.let{
                    onLocationUpdate(
                        poi.copy(latitude = it.latitude, longitude = it.longitude)
                    )}},
                    modifier = Modifier.weight(1f),
                    colors = ButtonDefaults.buttonColors(containerColor =
                                                            Color(0xFF4CAF50))
                ){
                    Text("Popravi GPS", fontSize = 10.sp)
                }
```

```
                Button(onClick = {editingPoi = poi},modifier = Modifier.weight(1f)){
                    Text("Izmeni", fontSize = 10.sp)
                }
                IconButton(onClick = {poiToDelete = poi}){
                    Icon(Icons.Default.Delete, tint = Color.Red)
                }
```

Подаци су додатно осигурани кроз систем дијалога за потврду (*AlertDialog*). Свака деструктивна акција, попут брисања тачке, захтева потврду, док *BackHandler* омогућава контролисано напуштање панела уз проверу намерног изласка. Поред основних акција овај модул садржи и дијалоге за уређивање описа.

4.3 Реализација серверског система на .NET 8 платформи

Серверски систем представља ослонац целе архитектуре, обезбеђујући податке за AR модул и мапе, као и управљање физичким ресурсима на серверу. Систем је реализован у развојном окружењу *Visual Studio* коришћењем .NET 8 платформе и структуиран је према принципима чисте архитектуре. Овакав приступ омогућава јасно раздвајање пословне логике од базе података и спољних сервиса.



Слика 4.1 – Приказ структуре засноване на чистој архитектури

4.3.1 Доменски модели и техничка прецизност

Унутар датотеке *Domain.Models* дефинисан је ентитет *POI.cs.* За разлику од стандардних система где се за децималне вредности често користи тип *decimal* или *float,* у

овом решењу је за координате *Latitude* и *Longitude* имплементиран тип *double.* Овај тип је изабран јер обезбеђује између 15 и 17 децимала, што омогућава велику GPS прецизност. Таква стабилност података је неопходна како би се осигурало њихово тачно постављање у 3D простору у односу на физичку локацију корисника. Поред нумеричких вредности, модел садржи референце ка категорији објекта, што кроз *virtual* својства омогућава ефикасно повезивање са 3D ресурсима.

```
//Domain/Models/POI.cs
public class POI
{
    public required Guid Id {get; init;}
    public required string Name {get; set;}
    public required double Latitude {get; set;}
    public required double Longitude {get; set;}
    public string? WebsiteUrl {get; set;}
    public int? CategoryId {get; set;}
    public virtual Category? Category {get; set;} = null!;
}
```

4.3.2 Апстракција приступа подацима и *Repository* образац

Како би се постигла максимална независност слојева, приступ бази је реализован коришћењем *Repository* обрасца. Први корак у овој реализацијиа је дефинисање такозваног уговора кроз спрегу *IPOIRepository.cs,* који дефинише потребне методе за CRUD операције без откривања детаља о самој бази података. Апликативни слој комуницира искључиво са овом спрегом, што омогућава лаку замену технологије складиштења у будућности без утицаја на пословну логику.

Стварна имплементација се налази у слоју инфраструктуре унутар класе *POIRepository.cs*. Ова класа користи *Entity Framework Core* за директну интеракцију са SQL сервером. Посебно је важна употреба *.Include(p => p.Category)* методе унутар упита за преузимање података, што представља механизам „ нестрпљивог учитавања “ (енгл. *Eager loading*). На овај начин се у оквиру само једног упита ка бази преузимају не само координате тачке, већ и сви метаподаци о категорији, укључујући називе 3D модела и иконица, што значајно убрзава одзив саме апликације.

```
//Domain/Interfaces/IPOIRepository.cs
public interface IPOIRepository {
    Task<IEnumerable<POI>> GetAllPOIs();
    Task<POI?> GetPOIbyID(Guid id);
    Task AddPOI (POI poi);
    Task DeletePOI (POI poi);
    ...
```

```
//Infrastructure/Repository/POIRepository.cs
public class POIRepository : IPOIRepository{
    private readonly IApplicationDbContext _context;
    public POIRepository (IapplicationDbContext context){
        _context = context
    }
    public async Task <POI?> GetAllPOIs()
    {
        return await _context.POIs.Include(p => p.Category).ToListAsync();
    }
    public async Task AddPOI(POI poi){
        await _context.POIs.AddAsync(poi);
        await _context.SaveChangesAsync();
    }
    //Остале CRUD методе
}
```

4.3.3 CQRS образац и DI механизам

За обраду комплексних захтева коришћен је CQRS образац подржан *MediatR* библиотеком. Целокупан систем заснован је на механизму убризгавања зависности (енгл. *Dependecy Injection* - DI), где се репозиторијум не инстанцира ручно, већ се „ убризгава “ кроз конструкторе *Handler* – а. Ово омогућава чистији код и лакше тестирање сваке појединачне компоненте система.

Посебан технички изазов решен је у оквиру команди и упита, као што су *UpdatePOICommandHandler* и *GetPOIDescriptionQueryHandler*. Пошто систем користи хибридни модел складиштења (база за податке и систем датотека за описе), *Handler* мора осигурати њихову синхронизацију. Уколико администратор промени име објекта, систем кроз *Directory.Move* методу аутоматски преименује датотеку на диску сервера. Овај процес укључује и чишћење улазних података од невалидних карактера за систем датотека, чиме се спречавају грешке приликом приступа ресурсима и осигувара сигурност података.

```
//Application/Commands/UpdatePOICommandHandler.cs
public class UpdatePOICommandHandler : IRequestHandler<<UpdatePOICommand>
{
    private readonly IPOIRepository _repository;
    private readonly string _filesPath;
    public UpdatePOICommandHandler(IPOIRepository repoistory,
                                   IConfiguration configuration)
    {
        _filesPath = configuration["FileSettings:PoisDescriptionPath"]!;
        _repository = repository;
    }
```

```
    public async Task Handle(UpdatePOICommand request,CancellationToken
                                cancellationToken)
    {
            var poi = await _repository.GetPOIbyID(request.Id);
            if(poi == null) throw new Exception("POI nije pronadjen u bazi");
            //Припрема валидних имена за систем датотека
            string oldSafeName = string.Concat(poi.Name
                                                .Split(Path.GetInvalidFileNameChars())
                                               )
        string newSafeName = string Concat(request.Name
                                                .Split(Path.GetInvalidFileNameChars())
                                      )
        string oldFolderPath = Path.Combine(_filesPath, oldSafeName);
        string newFolderPath = Path.Combine(_filesPath, newSafeName);
        //Логика за синхронизацију датотека на серверу
        if(oldSafeName != newSafeName && Directory.Exists(oldFolderPath)){
            if(Directory.Exists(newFolderPath)){
                Director.Delete(newFolderPath, true);
             }
             Directory.Move(oldFolderPath, newFolderPath);
         }else if(!Directory.Exists(newFolderPath)){
            Directory.CreateDirectory(newFolderPath);
         }
         //Ажурирање ентитета и перзистенција у SQL бази
        poi.Name = request.Name;
        poi.Latitude = request.Latitude;
        poi.Longitude = request.Longitude;
        ...
        await _repository.UpdatePOI(poi);
    }
}
```

4.3.4 Конфигурација перзистенције кроз *Fluent API*

Унутар *ApplicationDbContext.cs* реализована је конфигурација базе података коришћењем *Fluent API* – ја. Уместо ослањања на подразумевана подешавања, овим путем су стриктно дефинисана ограничења попут максималне дужине поља, што штити базу од непланираног заузећа простора и оптимизује перформансе претраге. Такође, дефинисан је *DeleteBehavior.Restrict* на страним кључевима, који спречава брисање категорија уколико у систему постоји бар једна тачка везана за ту категорију, чиме се елиминише могућност појаве грешке у AR приказу због недостајућих модела. Додатно је реализован механизам за почетно пуњење базе (*Seeding*), који при првој миграцији аутоматски генерише основне категорије објеката са свим припадајућим називима ресурса.

```
//Infrastructure/Persistence/ApplicationDbContext.cs
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.onModelCreating(modelBuilder);
    modelBuilder.Entity<POI>(entity =>
    {
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Name)
            .IsRequired()
            .HasMaxLength(200);
        //Остали параметри на сличан начин
        //Дефинисање релације и рестрикција брисања
        entity.HasOne(p => p.Category)
              .WithMany(c => c.POIs)
              .HasForeignKey(p => p.CategoryId)
              .onDelete(DeleteBehavior.Restrict);
    });
    //Почетно пуњење базе категоријама
    modelBuilder.Entity<Category>().HasData(
        new Category{
            Id = 1,
            Name = "Fakultet",
            ModelFileName = "fakultet.glb",
            IconName = "ic_fakultet"
        } ...
    )
}
```

4.3.5 Презентациони слој и преглед API путања

Презентациони слој система чини *POIController.cs* који представља серију REST приступних тачака. Комуникација са мобилним корисником се врши искључиво путем JSON формата, који омогућава ефикасну десеријализацију објеката на *Android* платформи. Свака путања је дефинисана да одговара специфичним функцијама апликације, од обичног излиставања тачака до сложених операција и асинхорног преузимања текстуалних описа са сервера. У следећој табели приказана је коначна структура API спреге:

Метода	Путања	Опис акције и намена
GET	/POIList	Преузимање комплетне листе тачака са координатама и метаподацима за AR рендеровање.

POST	/CreatePOI	Креирање нове тачке од интереса и генерисање датотеке за текстуалне описе на серверу
PUT	/UpdatePOI/{id}	Комплетна измена података о објекту (назив, вебсајт, категорија) уз синхронизацију датотека
PUT	/UpdateLocation/{id}	Специјализована рута за брзу промену координата директно са терена
DELETE	/Delete/{id}	Трајно уклањање објекта из базе
GET	/GetDescription/{id}	Асинхроно учитавање садржаја текстуалне датотеке са описом локације и слање клијенту
PUT	/UpdateDescription/{id}	Измена текстуалног садржаја унутар датотеке описа на серверу
GET	/GetCategories	Преузимање категорија за потребе филтрирања унутар апликације
POST	/AdminPassword	Сигурносна провера администраторске лозинке (из appsettings.json конфигурације) за приступ панелу за управљање

Табела 4.1 – Преглед имплементираних REST приступних тачака на серверу

4.4 Реализација хибридног приказа

Основа техничког решења заснива се на хибридном систему који кориснику нуди две комплементарне перспективе: високу прецизност локалне оријентације кроз проширену стварност и шири просторни контекст путем интерактивне мапе. Највећи технички изазов оваквог приступа лежи у захтеву за математичком тачношћу при израчунавању позиција објеката, као и у њиховој константној синхронизацији у реалном времену како би се осигурала усклађеност података на оба приказа.

Реализација овог хибридног модела захтева алгоритме за комбиновање података са више сензора уређаја истовремено. Док мапа користи стандрадне пројекције за приказ локација на 2D равни, AR модул мора да трансформише географске координате у локални 3D простор у односу на тренутну позицију и оријентацију корисника.

4.4.1 Реализација мрежне комуникације коришћењем *Retrofit* библиотеке

За потребе динамичког преузимања података са удаљеног сервера, у *Android* апликацији је реализована библиотека *Retrofit 2.* Она представља типски безбедан HTTP клијент који омогућава дефинисање API рута путем интерфејса, чиме се унапређује читљивост кода и олакшава одржавање система [30].

Основу реализације чини интерфејс *PoiApiService*, у којем су дефинисане методе за све кључне операције. Свака метода користи одговарајући HTTP глагол (*@GET, @POST, @PUT, @DELETE*) који одговара приступним тачкама дефинисаним на серверској страни.

```
//data/PoiApiService.kt
interface PoiApiService {
    @GET("POIList")
    suspend fun getAllPois():Response <List<PoiDto>>
    @POST("AdminPassword")
    suspend fun verifyAdmin(@Body command:VerifyAdminCommand)
                            :Response<AdminResponse>
    @PUT("UpdateCoordinates/{id}")
    suspend fun updatePoiLocation(@Path("id") id: String,@Body poi:PoiDto)
                            :Response<Unit>
    //Остале методе ...
}
```

Главне карактеристике ове реализације обухватају:

- Асинхрони рад – Употребом *suspend* функција и *Kotlin Coroutines* механизма, мрежни захтеви се извршавају у позадинској нити, чиме се спречава блокирање корисничког интерфејса и осигурава течан рад апликације.
- Аутоматска десеријализација – Интеграцијом са *Gson* декодером, JSON одговори са сервера се директно мапирају у *POIDto* објекте спремне за даљу обраду у AR модулу.
- Ефикасно читање ресурса – За преузимање описа локација коришћен је тип *ResponseBody,* који омогућава директан пренос сировог садржаја текстуалних датотека без потребе за додатним моделима података.

Овако структуиран комуникациони слој представља техничку основу система, обезбеђујући апликацији прецизне и ажурне податке.

4.4.2 Функционалност проширене стварности

Имплементација AR модула заснива се на *ARCore* оквиру и концепту „*Location - based AR*“, где се виртуелни објекти везују за стварне географске координате уместо за детектоване површине. Овакав приступ захтева високу прецизност GPS сигнала и сензора оријентација како би се обезбедило тачно просторо позиционирање виртуелних објеката.

Рад модула дефинисан је кроз динамичко управљање AR сесијом и хијерархијом сцене. Софтверска логика константно надзире техничке параметре – нагиб уређаја, спремност сесије и прецизност локације – и активира рендеровање маркера само када су сви предуслови испуњени.

4.4.2.1 Управљање дозволама и приступ сензорима

С обзиром да је за функционалност проширене стварности неопходан приступ камери, а за рад мапе прецизна GPS локација, реализован је систем за управљање дозволама. У оквиру *Jetpack Compose* окружења ово је реализовано коришћењем *rememberLauncherForActivityResult* механизма, који омогућава апликацији да асинхроно реагује на одлуку корисника.

```
//PermissionManager.kt
val cameraLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestPermission()
){granted -> cameraGranted = granted}

val locationLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestPermission()
){granted -> locationGranted = granted}
```

При сваком покретању апликације, систем проверава да ли су дозволе већ додељене. Уколико нису, активирају се објекти за покретање активности (енгл. *Launcher*) који покрећу системски дијалог за корисника. Ако корисник одбије неку од дозвола, апликација не прелази на главни кориснички интерфејс, већ приказује информативни екран са поруком да је потребно одобрити приступ сензорима и камери. Тиме се спречава покушај иницијализације *ARCore* сесије над затвореном камером, што би довело до пада апликације.

Овај приступ осигурава да су сви неопходни ресурси физичке архитектуре доступни пре почетка обраде података са сензора и рендеровања 3D маркера.

4.4.2.2 Конфигурација *ARCore* сесије и подешавање сцене

Основа за рендеровање проширене стварности је правилно конфигурисана AR сесија која дефинише како ће уређај приказивати податке са камере и сензора. У нашој

реализацији, функција *setupARCore* врши иницијализацију сесије уз специфична подешавања за спољашњу навигацију.

```
fun setupARCore (context: Context, arSceneView: ArSceneView){
    val session = Session(context)
    val config = Config(session).apply{
        planeFindingMode = Config.PlaneFindingMode.HORIZONTAL
        updateMode = Config.UpdateMode.LATEST_CAMERA_IMAGE
        lightEstimationMode = Config.LightEstimationMode.AMBIENT_INTESITY
        focusMode = Config.FocusMode.AUTO
    }
    session.configure(config)
    arSceneView.apply{
        setupSession(session)
        planeRenderer.isVisible = true
        planeRenderer.isEnabled = true
        scene.addOnUpdateListener{frameTime ->
            val frame = arFrame ?: return @addOnUpdateListener
            val pointCloud = frame.acquirePointCloud()
            pointCloud.release()
        }
    }
}
```

Главни елементи ове конфигурације су :

- *UpdateMode.LATEST_CAMERA_IMAGE* – осигурава да се виртуелни објекти ажурирају синхроно са видео преносом, чиме се смањује кашњење приказа
- *PlaneRenderer* – Омогућава кориснику визуелну потврду да систем успешно детектује физички простор.
- *PointCloud* процесуирање – кроз *addOnUpdateListener*, систем у сваком фрејму врши преузимање облака тачака. Иако се примарна оријентација маркера ослања на GPS и азимут, континуирана обрада облака тачака омогућава *ARCore* – у да прецизније прати ситне покрете уређаја и спречи такозвано шетање маркера по екрану.
- *FarClipPlane* – Вредност удаљености исцртавања је унутар *ArSceneView* конфигурације повећана на 70 метара, како би се омогућио приказ удаљених локација од интереса које би иначе биле одсечене стандардним подешавањима.

4.4.2.3 Обрада података са сензора и софтверска калибрација азимута

Реализација стабилног *Location-based AR* система захтева напредну обраду сигнала из GPS пријемника и инерцијалних сензора (жироскопа, акцелерометра и магнетометра).

Главни проблем представља несавршеност ових извора, попут шума GPS сигнала или магнетних сметњи што узрокује дрхтање и померање виртуелних објеката.

Ово поглавље описује архитектуру *ViewModel* слоја који врши прикупљање и корекцију података. Процес обухвата примену *Low-Pass* филтера за глатко кретање, фузију сензора за прецизну оријентацију, као и реализацију специфичне методе референтног азимута за софтверску калибрацију правца у реалном времену.

Очитавање и стабилизација локације – Класа *LocationViewModel* представља примарни извор података, за позиционирање у AR сцени. Коришћењем *FusedLocationProviderClient* библиотеке, систем захтева податке највишег приоритета уз конфигурацију оптимизовану за динамичко окружење:

```
val request = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 500L)
        .setMinUpdateIntervalMillis(200L) //Најбржи интервал ажурирања
        .setMinUpdateDistanceMeters(0f) //Реакција на свако померање
        .setWaitForAccurateLocation(false) //Спречава кашњење прве локације
        .build()
```

Због нестабилности GPS сигнала, имплементиран је двостепени процес обраде. Први корак обухвата иницијализацију при првом очитавању, док се у сваком наредном кораку примењује *Low-Pass* филтер за стабилизацију координата. Логика првог очитавања осигурава правилну иницијализацију почетних координата пре самог процеса филтрирања:

```
//Иницијализација при првом покретању
override fun onLocationResult(result: LocationResult) {
    result.lastLocation?.let{ loc ->
        if (_location.value == null) {
            lastLat = loc.latitude
            lastLon = loc.longitude
            viewModelScope.launch {
                _location.value = loc
            }
            return
        }
//Low-Pass филтер
private fun lowPassFilter(currentValue: Double, alpha: Float, lastValue: Double):
Double {
    return lastValue + alpha * (currentValue - lastValue)
}
//Примена филтера
val stabilizedLat = lowPassFilter(loc.latitude, gpsAlpha, lastLat)
val stabilizedLon = lowPassFilter(loc.longitude, gpsAlpha, lastLon)
```

Овакав приступ омогућава да *StateFlow* пренос испоручује стабилизоване вредности, чиме се постиже глатко кретање AR маркера кроз физички простор без визуелног скакања.

Фузија сензора и стабилизација оријентације – За прецизно одређивање правца гледања корисника користи се фузија сензора путем *TYPE_ROTATION_VECTOR*. Овај приступ је много бољи у односу на обичан компас јер комбинује податке жироскопа и аклцелерометра како би се елиминисале магнетне сметње. Кључни део реализације унутар *OrientationViewModel* класе је прилагођавање координатног система уређаја. Како се апликација користи у моду проширене стварности тако да се уређај држи усправно а не равно, неопходно је извршити ремапирање оса како би се азимут правилно израчунао.

```
//Израчунавање матрице ротације и ремапирање за усправни положај
SensorManager.getRotationMatrixFromVector(rotationMatrix, event.values)
SensorManager.remapCoordinateSystem(
    rotationMatrix,
    SensorManager.AXIS_X, //X остаје иста
    SensorManager.AXIS_Z, //Z постаје нова вертикална
    remappedMatrix
)
//Издвајање азимута из ремапиране матрице
val orientation = FloatArray(3)
SensorManager.getOrientation(remappedMatrix, orientation)
val az = Math.toDegrees(orientation[0].toDouble()).toFloat()
val normalizedAz = (az + 360) % 360
```

Након добијања сировог азимута, примењује се специфичан *Low-Pass* филтер који решава проблем кружног прелаза (енгл. *Heading Wrap*). Овај филтер омогућава да сцена не ротира нагло када корисник пређе границу од 360/0 степени:

```
private fun headingWrapFilter(currentAzimut : Float):Float{
    val alpha = 0.15f
    var delta = currentAzimut - lastFilteredAzimut
    if(delta > 180) delta- = 360 else if(delta < -180) delta+ = 360
    val newFilteredAzimut = lastFilteredAzimut + delta*alpha
    lastFilteredAzimut = (newFilteredAzimut + 360) % 360
    return lastFilteredAzimut
}
```

Цео процес се извршава са *SENSOR_DELAY_UI* интервалом, што пружа оптимизацију између потрошње батерије и брзине одзива AR сцене. Стабилизовани подаци се након тога кроз *StateFlow* прослеђују компоненти за исцртавање, чиме се постиже течан приказ маркера.

Стабилизација правца референтним азимутом – Како би се спречило нагло померање AR сцене услед варијације сензора, реализовано је иницијално фиксирање

правца. Коришћењем *LaunchedEffect* механизма, апликација приликом покретања фиксира први очитани стабилизовани азимут као референтну тачку.

```
LaunchedEffect(isInitializing){
    if(!isInitializing && referenceAzimut == null){
        referenceAzimut = azimut //фиксирање почетног правца
    }
}
```

Просторна трансформација и тригонометријски прорачуни – Трансформација географских координата у AR простор ослања се на два кључна прорачуна: растојање и правац. За израчунавање растојања између корисника и објекта користи се Хаверсинусна формула (енгл. *Haversine Formula*) [29], која узима у обзир закривљеност земље. Уместо класичне дистанце, ова формула обезбеђује прецизност на већим удаљеностима:

$$d = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

Где су ϕ и λ географска ширина и дужина, а r полупречник Земље. У реализацији је коришћена вредност r = 6371km.

Да би се одредио угао под којим се маркер поставља у односу на север, користи се функција за израчунавање правца. Код ове функције је од кључне важности за правилно позиционирање маркера лево или десно од корисника:

```
fun bearingToPOI(userLat:Double, userLon:Double, poiLat:Double, poiLon:Double)
:Double{
    val lat1 = Math.toRadians(userLat)
    val lon1 = Math.toRadians(userLon)
    val lat2 = Math.toRadians(poiLat)
    val lon2 = Math.toRadians(poiLon)

    val dLon = lon2 - lon1
    val y = sin(dLon) * cos(lat2)
    val x = cos(lat1) * sin(lat2) - sin(lat1) * cos(lat2) * cos(dLon)
    //Резултата у опсегу -180 до 180 степени претворен у 0 до 360
    return (Math.toDegrees(atan2.(y,x)) + 360) % 360
}
```

Финална фаза је трансформација координата у 3D простор кроз функцију *CalculateMarkerPosition*. На основу разлике између азимута уређаја и правца ка циљу израчунава се релативни угао, који се нормализује на опсег од -180 до 180 степени. Коришћењем тригонометријских функција и стварне удаљености у метрима врши се пројекција на X и Z осе AR сцене. Висина маркера је фиксирана пола метра ниже у односу

на камеру уређаја. Овим прорачуном се GPS подаци претварају у *Vector3* координате, чиме се омогућава постављање виртуелног објекта на тачну локацију у стварном свету.

```
fun CalculateMarkerPosition(
    referenceAzimut:Float,
    poiBearing:Double,
    distanceMeters:Float
):Pair<Vector3, Float>{
    var relativeAngle = (poiBearing - referenceAzimut).toFloat()
    if(relativeAngle > 180){
        relativeAngle -= 360
    }else if(relativeAngle < 180){
        relativeAngle += 360
    }
    val angleToRadians = Math.toRadians(relativeAngle.toDouble())
    val x = sin(amgleToRadians) * distanceMeters
    val z = -cos(angleToRadians) * distanceMeters
    val y = -0.5f
    val position = Vector3(x.toFloat(), y, z.toFloat())
    return position to 0.8f
}
```

4.4.2.4 Рендеровање 3D маркера у реалном времену

Процес визуелизације објеката у проширеној стварности представља финални корак у коме се израчунате координате трансформишу у видљиве 3D моделе унутар *ArSceneView* компоненте. Како би систем радио течно, реализовани су механизми асинхроног учитавања и паметног кеширања који спречавају преоптерећење физичке архитектуре.

Функција *addModel* задужена је за учитавање GLB датотека ван главне нити апликације, чиме се избегава могућност замразавања корисничког интерфејса. Коришћењем *renderablesCache* мапе, систем чува већ учитане моделе и креира њихове копије у меморији, што убрзава поновно приказивање истог маркера.

```
fun addModel(context:Context,arSceneView:ArSceneView,modelFileName:String,
    onNodeCreated:(Node)->Unit,onError:(Throwable)->Unit){
        //Провера да ли модел већ постоји у кеш меморији
        val existingRenderable = renderableCaches[modelFileName]
        if(existingRenderable != null){
            onNodeCreated(Node().apply{renderable = existingRenderable.makeCopy()})
            return
        }
        //Дефинисање путање до 3D модела и асинхроно креирање
        val renderableSource = RenderableSource.builder()
            .setSource(context, "models/$modelFileName".toUri(),
                RenderableSource.SourceType.GLB)
```

```
            .setRecenterMode(RenderableSource.RecenterMode.ROOT)
            .build()
        ModelRenderable.builder.setSource(context, RenderableSource).build()
            .thenAccept{renderable ->
                renderableCache[modelFileName] = renderable
                onNodeCreated(Node().apply{this.renderable = renderable.makeCopy()})
            }
            .exceptionally{throwable -> onError(throwable);null}
```

Функција *addMarker* управља безбедним додавањем објеката тако што проверава статус AR сесије и спречава дуплирање захтева за исти маркер. Израђен је и сигурносни бројач који након 10 секунди ресетује систем у случају грешке, чиме се гарантује да апликација остане респонзива чак и при лошим условима рада.

```
fun addMarker(context: Context, arSceneView: ArSceneView, worldAnchorNode:
            AnchorNode, poi: PoiDto, position: Vector3, scale: Float,
            markerNodes: MutableMap<String, Node>, loadingMarkers:
            MutableSet<String>) {
    // Спречавање креирања ако маркер већ постоји или је у процесу учитавања
    if (markerNodes.containsKey(poi.id) || loadingMarkers.contains(poi.id)) return
    loadingMarkers.add(poi.id)
    isProcessingModel = true
    //Сигурносни бројач
    val safetyTimeout = Runnable {
        if (loadingMarkers.contains(poi.id)) {
            loadingMarkers.remove(poi.id)
            isProcessingModel = false
        }
    }
    handler.postDelayed(safetyTimeout, 10000)
    addModel(context, arSceneView, poi.modelFileName, onNodeCreated = { node ->
// Везивање новог чвора за референтну тачку и подешавање локалних трансформација
        node.setParent(worldAnchorNode)
        node.localPosition = position
        node.localScale = Vector3(scale, scale, scale)
        markerNodes[poi.id] = node
        loadingMarkers.remove(poi.id)
        isProcessingModel = false // Ослобађање процеса за следећи модел
    }, onError = {
        loadingMarkers.remove(poi.id)
        isProcessingModel = false
    })
}
```

Унутар главне петље рендеровања, систем континуирано ажурира стање свих активних маркера како би они деловали као фиксни део стварног простора. Коришћењем линеарне интерполације, позиције маркера се глатко мењају у складу са кретањем

корисника, чиме се избегава визуелно сецкање које изазивају варијације у прецизности GPS сигнала. Такође, примењује се динамичко скалирање, које удаљеније маркере чини пропорционално већим ради бољег визуелног искуства.

```
//Ажурирање трансформација маркера унутар сваког фрејма
if(existingNode.isEnabled){
    val currentPos = existingNode.localPosition
    //Рачунање разлике између тренутне и нове позиције
    val diff = Vector3.subtract(absoluteTargetPos, currentPos).length()
    //Линеарна интерполација - фактор 0.15 омогућава да маркер клизи ка правој
     позицији
    //Ако је GPS скок већи од 5м маркер се одмах пребацује на нову позицију
    existingNode.localPosition = if(diff > 5.0f){
        absoluteTargetPos
    }else{
        Vector3.lerp(currentPos, absoluteTargetPos, 0.15f)
    }
    //Динамичко скалирање које омогућава видљивост маркера на даљини
    val scale = (0.5f + (distance / 100f) * 1.5f).coerceIn(0.5f, 2.0f)
    existingNode.localScale = Vector3(scale, scale, scale)
    //Поравнање ротације маркера са ротацијом камере
    existingNode.worldRotation = cameraRotation
}
```

4.4.3 Реализација интерактивне мапе

На основу претходно дефинисаног концепта хибридног интерфејса са подељеним екраном, техничка реализација мапе обухвата интеграцију *Osmdroid* библиотеке и *Jetpack Compose* окружења. Фокус овог модула је на прецизном мапирању географских података у реалном времену и обезбеђивању високих перформанси приликом истовременог рада AR сесије и приказа мапе. Систем је пројектован тако да кроз асинхроне процесе синхронизује стање филтера и локације између оба визуелна сегмента апликације.

4.4.3.1 Логика приказа и аутоматско центрирање локације

Након извршене основне конфигурације, фокус имплементације прелази на управљање приказом унутар *Jetpack Compose* окружења. Коришћењем *AndroidView* компоненте, омогућена је интеграција *MapView* објекта који подржава напредне функције попут интеракције вишеструким додирима (енгл. *Multi-Touch*), контрола за зумирање и детекције догађаја на мапи. Кључни део је и корисничко искуство при првом отварању мапе, где систем аутоматски анимира приказ ка кориснику ако је GPS прецизност задовољавајућа.

Додатно реализован је *MapEventsOverlay* који омогућава да се приликом додира на било који празан део мапе аутоматски затворе сви активни информациони прозори маркера. Ово омогућава чистији преглед и спречава нагомилавање визуелних елемената током претраге мапе.

```
//Аутоматско центрирање на локацију корисника
LaunchedEffect(currentLoaction){
    currentLocation?.let{loc ->
        //Провера прецизности сигнала
        if(loc.accuracy < 100f){
            val userPoint = GeoPoint(loc.latitude,loc.longitude)
            //Центрирање при првом учитавању или ако се корисник много удаљио
            if(!hasCenteredOnce || distance > 10000.0){
                mapView.controller.animateTo(userPoint)
                mapView.controller.setZoom(17.5)
                hasCenteredOnce = true
            }
        }
    }
}
```

4.4.3.2 Креирање и динамичко ажурирање маркера

Процес постављања тачака од интереса реализован је кроз управљање слојевима мапе како би се очувале перформансе уређаја. Уместо поновног рендеровања свих маркера при свакој промени, апликација врши подељено ажурирање – уклањају се само они маркери чији идентификатори више нису у листи података, док се постојећи ажурирају новим координатама или удаљеностима.

```
//Динамичко мапирање података и прорачун удаљености
poiList.forEach{poi ->
    //Проналажење постојећег маркера у кешу или иницијализација новог
    val marker = poiMarkers.getOrPut(poi.id){
        Marker(mapView).apply{
            setAnchor(Marker.ANCHOR_CENTER, Marker.ANCHOR_BOTTOM)
            mapView.overlays.add(this)
        }
    }
    //Форматирање удаљености у метрима или километрима
    val distanceText = if(distance < 1000) "${distance.toInt()} m"
                       else "%.1f km".format(distance / 1000f)
    marker.title = poi.name
    marker.snippet = "Razdaljina: $distanceText"
    marker.icon = ContextCompat.getDrawable(context, getIconResId(poi.iconName))
    marker.position = GeoPoint(poi.latitude, poi.longitude)
}
```

4.5 Реализација административног модула и управљања подацима

Администартивни модул представља технички слој апликације задужен за директну манипулацију подацима у реалном времену. Поред измене постојећих, систем омогућава и креирање потпуно нових тачака од интереса, ослањајући се на прецизне податке са сензора уређаја.

4.5.1 Механизам прикривене аутентификације

Техничка реализација овог слоја заштите заснива се на дискретној детекцији корисничких интеракција помоћу променљиве стања *mutableIntStateOf*. Програмска логика увећава овај бројач при сваком притиску на специфичан елемент наслова, а када вредност достигне пет кликова, апликација мења логичко стање и активира *AlertDialog* за други степен провере. Унутар дијалога се врши валидација унете лозинке у односу на дефинисани кључ. Уколико је лозинка тачна отвара се администраторски интерфејс а грешка или затварање прозора ресетује бројач на нулу.

4.5.2 Логика креирања нових тачака од интереса

За додавање нових локација развијен је *PoiFormDialog* који обједињује унос података са GPS очитавањем. Приликом креирања новог објекта, систем генерише привремени јединствени идентификатор (*GUID*) и поставља почетне вредности категорија из *categories* листе. Важан део логике је праћење прецизности параметра у реалном времену. Систем бојама сигнализира администратору квалитет сигнала, чиме се осигурава да се нова тачка упише у базу само када је GPS позиција довољно прецизна.

```
//Креирање новог POI објекта са тренутним координатама
val finalPoi = PoiDto(
    id  =  "00000000-0000-0000-0000-000000000000",  //Генерички  ID  за  нови  унос
    name = name,
    latitude = latText.toDoubleOrNull() ?: 0.0,
    longitude = lonText.toDoubleOrNull() ?: 0.0,
    websiteUrl = webUrl,
    //... остали параметри
)
```

4.5.3 Динамичко управљање стањима

Срж модула чини напредно управљање стањима путем *mutableStateOf* променљивих. Свака измена координата користи *copy* методу над *PoiDto* моделом. Ово омогућава да дугме за калибрацију моментално пребрише старе координате тренутном локацијом

администратора, истовремено чувајући остале податке о објекту непромењеним. Систем такође користи меморијски оптимизовану претрагу која у реалном времену филтрира листу на основу назива или категорије.

5. Испитивање решења и резултати

Након успешно завршене реализације клијентског и серверског дела апликације, прешло се на фазу испитивања у реалном окружењу. У овом делу рада анализира се понашање система у пракси, при чему је фокус стављен на верификацију кључних функција, стабилност мрежне комуникације и прецизност приказа AR објеката. Тестирање је извршено на паметном мобилном уређају *Samsung Galaxy S23* (*Android 14*), који захваљујући сензорским карактеристикама омогућава пуну функционалност *ARCore* платформе.

С обзиром да рад саме апликације зависи од константне везе са базом података, први изазов током тестирања на локацијама изван локалне мреже представљао је приступ локалној бази података односно серверу. Овај проблем је превазиђен коришћењем *ngrok* сервиса, којим је омогућено креирање сигурног тунела ка серверу.

5.1 Испитивање функционалности система

Тестови су приказани у табели, која садржи технички опис и поступак извођења сваког теста, очекиване резултате, као и оцену успешности.

Опис теста	Очекивани резултати	Успешност
Опис: Управљање дозволама **Поступак:** Инсталација и прво покретање апликације уз одбијање, а затим прихватање дозвола за камеру и локацију	Систем онемогућава приступ главном екрану док се не одобре неопходне дозволе за рад сензора	Успешно

Опис: Иницијализација *ARCore* сесије **Поступак:** Након одобрења дозвола усмерити камеру ка отвореном простору	Камера се активира и систем успешно покреће праћење окружења без грешака	Успешно
Опис: Аутоматска калибрација севера **Поступак:** Покренути апликацију држећи телефон усправно у произвољном смеру	Систем успешно израчунава референтни азимут и поравнава AR свет са стварним севером без обзира на почетну оријентацију	Успешно
Опис: Преузимање података (ngrok/Retrofit) **Поступак:** Покренути апликацију уз активну интернет везу и ngrok тунел ка удаљеном серверу	Апликација успешно шаље захтев и прима JSON листу тачака од интереса из базе података	Успешно
Опис: Ограничење видљивости **Поступак:** Посматрати AR сцену док се корисник налази на удаљености већој од 20 метара од најближе тачке	Маркери остају скривени све док корисник не уђе у зону од 20 метара, чиме се спречава приказ са великим померајем	Успешно
Опис: Интеракција и приказ описа **Поступак:** Кликнути на исцртани 3D маркер објекта на екрану и на дугме детаљније	Отвара се дијалог са текстуалним описом који је преузет са сервера у формату текстуалне датотеке	Успешно

Опис: Функција калибрације координата **Поступак:** Отићи до локације, отворити админ панел и потврдити ажурирање координата	Тренутне GPS координате корисника/тестера се шаљу на сервер, база се ажурира и маркер се одмах поставља на нову позицију	Успешно
Опис: Визуелна стабилност (LERP, Low-Pass) **Поступак:** Ротирати телефон лево-десно и ходати ка тачки од интереса	Објекат не показује дрхтање при ротацији ни нагле промене позиције при промени GPS локације	Успешно
Опис: Интерактивна мапа и центрирање **Поступак:** Отворити мапу и кретати се, након тога кликнути на дугме за центрирање	Иконица човека прецизно прати кретање корисника, а мапа се након клика на дугме центрира	Успешно
Опис: Термална стабилност и FPS **Поступак:** Користити апликацију у AR режиму константно у трајању од 30 минута	Апликација задржава висок степен флуидности приказа и одзива интерфејса. Систем остаје оперативан упркос очекиваном загревању уређаја	Успешно

Табела 5.1 – Резултати тестирања апликације

5.2 Анализа и корекције делимично успешних тестова

Током тестирања на терену идентификовани су технички изазови који су захтевали додатне оптимизације програмске подршке како би се превазишла ограничења физичке архитектуре паметног мобилног уређаја.

Опис теста	Анализа	Корекција	Успешност
Опис: Визуелно позиционирање	Маркер се повремено појављује 3-4 метра бочно у односу на објекат услед несавршености GPS – а	Примена LERP функције која визуелно ублажава грешку GPS – а спречавањем наглих скокова маркера	Делимично успешно
Опис: Угаоно одступање	На растојањима преко 50 метара, минимална грешка компаса помера маркер десетинама метара у страну	Уведено ограничење приказа на радијус од 20 метара	Успешно
Опис: Шум сензора	Дрхтање AR објеката при стајању у месту због осетљивости магнетометра	Реализован *Low-pass* филтер за софтверско исправљање података са сензора	Успешно

Табела 5.2 – Анализа и корекција

Објашњење одступања GPS позиционирања – Приликом тестирања у реалном окружењу кампуса, примећено је да се маркери не појављују увек на идеалној позицији у односу на објекат (одступања су око 3 до 4 метра). Главни узрок овога је ограничење физичке архитектуре GPS пријемника паметних телефона који у близини објеката имају проблем рефлексије сигнала.

Иако се из базе података преузимају прецизне координате, GPS сензор уноси шум који резултује одступањем маркера од идеалне позиције. За потребе овог рада, проблем је ублажен применом LERP алгоритма. Ова метода не исправља грешку самог GPS – а, већ осигурава да маркер глатко иде ка својој правој позицији уместо да нагло мења положај.

6. Закључак

У оквиру овог рада успешно је реализована апликација за проширену стварност (AR) коришћењем *Android* платформе и *ARCore* технологије, са примарним циљем интеграције дигиталних података у реално физичко окружење. Апликација нуди корисницима интерактивно искуство које обухвата визуелизацију тачака од интереса, динамичко преузимање информација са удаљеног сервера и напредне административне функције за управљање подацима. *Android Studio* је изабран као основно развојно окружење због својих огромних могућности у управљању сензорима мобилног уређаја и директне подршке за *Google* – ове библиотеке намењене проширеној стварности.

Коришћењем .NET технологије на серверској страни, омогућена је стабилна позадинска инфраструктура која осигурава брз одзив и поузданост при раду са базом података.

Током развоја апликације, реализован је комплетан процес од иницијалне архитектуре система до крајњег испитивања на терену. Посебан акценат стављен је на решавање изазова који се тичу несавршености сензора физичке архитектуре, попут GPS – а и магнетометра. Кроз реализацију софтверских решења за филтрирање и интерполацију података, визуелна одступања су успешно сведена на минимум и постигнута је стабилност приказа маркера.

Апликација развијена у оквиру овог рада представља солидну основу за даљи развој сличних решења проширене стварности која могу наћи примену у широком спектру области. На пример, оваква решења се могу ефикасно користити у туризму за интерактивно вођење кроз знаменитости, у едукацији за визуелизацију историјских и научних података на терену, или у архитектури за приказ будућих објеката у стварном простору.

Даљи развој ове апликације могао би обухватити неколико праваца унапређења. Једна од значајних могућности је интерграција *Google Geospatial API* сервиса, чиме би се омогућило још прецизније позиционирање коришћењем визуелних података из окружења.

Такође, може се размотрити увођење сложенијих 3D модела и анимација, као и реализација механизама машинског учења за препознавање специфичних објеката у кадру камере. Оптимизација потрошње енергије и даље унапређење стабилности приказа остају важне ставке за будуће верзије система.

Технологија проширене стварности, као што је приказано у овом раду, пружа велики број могућности за превазилажење разлике између дигиталног и стварног света. Развој ове апликације представља значајан корак ка дубљем разумевању примене AR технологија на мобилним платформама, а потенцијал за даље унапређивање остаје велик. С обзиром на константан напредак у могућностима физичке архитектуре паметних телефона, очекује се да ће будућа решења донети још богатија искуства, отварајући потпуно нове правце за интеракцију корисника са подацима који га окружују.

7. Литература

[1] Azuma, R. T. „ A survey of augmented reality “, Presence: Teleoperators & Virtual Environments, 6(4), 355-385, 1997.

[2] Schmalstieg, D., Höllerer, T., „ Augmented Reality: Principles and Practice “, Addison-Wesley Professional, 2024.

[3] Lang, M. , „ Mobile Sensors and Tracking in Augmented Reality Systems “, Journal of Real-Time Image Processing, 18(2), 2023.

[4] Tussyadiah, I. P., et al., "The Illusion of Reality: Exploring Consumer Experience in Mobile Augmented Reality", 2023.

[5] Ivan Sutherland and Bob Sproull Create the First Virtual Reality Head Mounted Display System , https://www.historyofinformation.com/detail.php?id=861, 09.01.2026.

[6] Caudell, T.P., Mizzel, D.W., “Augmented reality : an application of heads-up display technology to manual manufacturing processes,” Hawaii International Conference on System Sciences, 1992.

[7] Rosenberg, L.B., „ Virtual Fixtures : Perceptual Overlays Enhance Operator Performance “, https://apps.dtic.mil/sti/citations/

[8] HitLab, „ History of AR Toolkit “, https://www.hitl.washington.edu/artoolkit/, 09.01.2026.

[9] Wikipedia : Pokémon Go, https://en.wikipedia.org/wiki/Pok%C3%A9mon_Go, 11.01.2026.

[10] Google Developers, „ Fundamental Concepts of ARCore “, https://developers.google.com/ar/discover/concepts, 11.01.2026.

[11] Influence of navigation parameters on cybersickness in virtual reality, https://dl.acm.org/doi/abs/10.1007/s10055-020-00474-2, 13.01.2026.

[12] Microsoft Learn „ .NET Documentation “, https://learn.microsoft.com/dotnet/ 13.01.2026

[13] Clean Architecture in .NET Core , https://medium.com/@jenilsojitra/clean-architecture-in-net-core-e18b4ad229c8m, 13.01.2026.

[14] Bogard, J. „ MediatR Documentation “, https://github.com/jbogard/MediatR , 13.01.2026.

[15] IBM Cloud Education, „ What is a REST API ? “ , https://www.ibm.com/think/topics/rest-apis , 14.01.2026.

[16] SmartBear Software. „ Swagger OpenAPI Specification “, https://swagger.io/ , 14.01.2026.

[17] Microsoft Learn. „ Entity Framework Core Overview “, https://learn.microsoft.com/ef/core/ 15.01.2026.

[18] How Visual Inertial Odometry (VIO) Works, https://www.thinkautonomous.ai/blog/visual-inertial-odometry/ , 18.01.2026.

[19] Google Developers, „ Working with Anchors “, https://developers.google.com/ar/develop/anchors , 18.01.2026.

[20] Google Developers, „ Sceneform SDK for Android (v1.15.0) “ – Reference Documentation , https://developers.google.com/sceneform/reference , 21.01.2026.

[21] Khronos Group, „ glTF 2.0 Specification “, https://www.khronos.org/gltf/ , 21.01.2026.

[22] Google Developers, „ Jetpack Compose: A modern design system for Android “, https://developer.android.com/jetpack/compose , 25.01.2026.

[23] Osmdroid Team, „ osmdroid: OpenStreetMap – Tools for Android “, https://github.com/osmdroid/osmdroid , 26.01.2026.

[24] OpenStreetMap Foundation, „ OpenStreetMap : The Free Wiki World Map “, https://www.openstreetmap.org , 26.01.2026.

[25] Android Open Source Project, „ Android System Architecture “, https://source.android.com/docs/core/architecture , 29.01.2026.

[26] Android Developers, „ The Activity Lifecycle Guide “, https://developer.android.com/guide/components/activities/activity-lifecycle , 29.01.2026.

[27] Android Developers, „ Sensors Overview and Best Practices “, https://developer.android.com/guide/topics/sensors/sensors_overview , 29.01.2026.

[28] Android Developers, „ Build Configuration overview “ , https://developer.android.com/build/gradle-build-overview , 01.02.2026.

[29] Esri support. Haversine Formula.ArcUser Online, https://community.esri.com/t5/coordinate-reference-systems-blog/distance-on-a-sphere-the-haversine-formula/ba-p/902128 , 10.02.2026.

[30] Square, „ Retrofit: A type-safe HTTP client for Android and Java “, https://square.github.io/retrofit/ , 15.02.2026.